

La structure et le code de Suivi_eleve

Document d'apprentissage. Il présente **l'organisation du projet** puis **explique le code ligne par ligne, sans répétition** : chaque construction (syntaxe, mécanisme, motif) est expliquée **une seule fois**, à sa première apparition, avec un renvoi (« voir §... ») quand elle revient.

Le projet compte environ **390 fichiers et 40 000 lignes**. Les fichiers **représentatifs** sont commentés ligne par ligne ; les autres fichiers suivent exactement les mêmes motifs : pour eux, on donne leur rôle et ce qu'ils ont de **particulier**. En lisant ce document dans l'ordre, vous serez capable de lire n'importe quel fichier du projet.

Lire d'abord `1-Technologies` pour savoir ce qu'est chaque outil.

Sommaire

- [Partie A — L'organisation du projet](#)
 - A.1 Arborescence commentée
 - A.2 Les fichiers de la racine
- [Partie B — Mémento de syntaxe TypeScript](#)
- [Partie C — L'API \(NestJS\)](#)
 - C.1 Le démarrage · C.2 La base (Prisma) · C.3 Les outils communs · C.4 L'authentification · C.5 Un module complet : les annonces · C.6 Le moteur de notifications · C.7 Les services transverses · C.8 Les autres modules · C.9 Les tests · C.10 L'image Docker de l'API
- [Partie D — Le site web \(Next.js\)](#)
- [Partie E — L'application mobile \(Expo\)](#)
- [Partie F — Le déploiement et la CI](#)
- [Partie G — Le voyage d'un message, de bout en bout](#)

Partie A — L'organisation du projet

A.1 Arborescence commentée

Suivi_eleve/	
├─ package.json	Scripts communs (pnpm dev:api, pnpm test...) et Prettier
├─ pnpm-workspace.yaml	Déclare les 3 projets du monorepo : api, web, mobile
├─ pnpm-lock.yaml	Versions exactes de tous les paquets (ne pas modifier à la main)
├─ .prettierrc	Règles de formatage (guillemets simples, virgules finales)
├─ .prettierignore	Fichiers que Prettier ne touche pas
├─ .editorconfig	Réglages d'éditeur (2 espaces, LF, UTF-8)
├─ .gitattributes	Fins de ligne LF, fichiers binaires
├─ .gitignore	Fichiers jamais envoyés sur GitHub (.env, node_modules...)
├─ .dockerignore	Fichiers jamais copiés dans les images Docker
├─ .env.example	Modèle des variables de configuration (développement)
├─ .env.production.example	Modèle des variables de production

- docker-compose.yml Développement : PostgreSQL + Redis
- docker-compose.prod.yml Production : 6 services (base, Redis, API, site, Caddy, sauvegarde)
- CLAUDE.md Règles du projet pour l'assistant IA
- README.md Présentation et démarrage rapide
- Help/ CES DOCUMENTS D'APPRENTISSAGE
- docs/
 - CAHIER_DES_CHARGES.md Ce que l'application doit faire (exigences EF-xx)
 - PROMPTS.md Les 18 étapes de développement
 - DEPLOIEMENT.md Mise en production sur un serveur
- deploy/
 - Caddyfile Configuration du serveur web frontal (HTTPS)
 - mettre-a-jour.sh Mise à jour du serveur en une commande
 - sauvegarde/ Image Docker de sauvegarde chiffrée (+ restauration)
- .github/workflows/ci.yml Vérifications automatiques sur GitHub
- api/ = LE SERVEUR (NestJS) =
 - package.json Paquets et scripts de l'API
 - Dockerfile Image de production
 - tsconfig.json Options du compilateur TypeScript
 - tsconfig.build.json Options pour la construction (sans les tests)
 - nest-cli.json Options de l'outil en ligne de commande de NestJS
 - .oxlintrc.json Règles du lint
 - vitest.config.ts Tests unitaires ; vitest.config.e2e.ts : tests de bout en bout
 - prisma7.config.ts Configuration de Prisma (schéma, migrations, seed, base)
 - prisma/
 - schema.prisma LES TABLES DE LA BASE
 - migrations/ Historique SQL des changements de tables (14 migrations)
 - seed.ts Données fictives de développement
 - test/ Tests de bout en bout (*.e2e-spec.ts)
 - src/
 - main.ts Point de départ : démarre le serveur
 - app.setup.ts Réglages communs (sécurité, préfixe /api, validation)
 - app.module.ts Module racine qui assemble tous les modules
 - app.controller.ts / app.service.ts Route /api/sante
 - generated/prisma/ Client Prisma GÉNÉRÉ (non versionné)
 - prisma/ Connexion à la base (PrismaService)
 - common/ Outils partagés : dates, pagination, pays, rôles, validation, Redis
 - auth/ Connexion, jetons, rôles, gardes, consentement
 - securite/ Limiteur de tentatives, IP de la requête
 - audit/ Journal des actions sensibles
 - stockage/ Fichiers (photos, pièces jointes)
 - sms/ Interface d'envoi direct d'un SMS (codes de connexion)
 - notifications/ MOTEUR D'ENVOI : règles, modèles, files, fournisseurs
 - plateforme/ Espace concepteur : écoles, abonnements, suspension
 - utilisateurs/ Comptes du personnel
 - classes/ Classes et année scolaire
 - tuteurs/ Parents / responsables
 - eleves/ Élèves, import/export Excel, données personnelles
 - appareils/ Appareils des élèves, QR codes, signalements
 - annonces/ Pas de cours, libération anticipée
 - absences/ Appel et absences des élèves
 - matieres/ Matières et enseignements
 - resultats/ Moyennes, bulletins PDF, décisions de fin d'année
 - comportements/ Comportements marquants
 - paiements/ Rappels de paiement et relances automatiques
 - evenements/ Réunions, sorties, fêtes (avec réponses des parents)
 - cli/ Scripts à lancer à la main (compte concepteur, clés VAPID)
- web/ = LE SITE (Next.js) =
 - package.json, next.config.ts, tsconfig.json, eslint.config.mjs, postcss.config.mjs, Dockerfile
 - proxy.ts Avant chaque page : renouvelle la session ou renvoie à /connexion
 - lib/ Outils : appel de l'API, session, types, pays, thème
 - components/ Composants réutilisables (menu, bandeau, formulaires, UI)

└─ public/	Fichiers servis tels quels : sw.js, hors-ligne.html, icônes
└─ app/	LES PAGES (une adresse = un dossier)
└─ layout.tsx	Mise en page racine (thème, service worker)
└─ page.tsx	« / » : redirige vers l'espace du rôle
└─ globals.css	Couleurs et mode sombre
└─ manifest.ts	Manifeste du site installable
└─ connexion/	Page de connexion + actions (parents et personnel)
└─ (admin)/	Espace du personnel : eleves, classes, absences, annonces...
└─ (parent)/	Espace des parents : /parent/...
└─ (plateforme)/	Espace concepteur : /plateforme/...
└─ telechargements/	Routes qui relaient un fichier (Excel, PDF, photo)
└─ mobile/	= L'APPLICATION MOBILE (Expo) =
└─ package.json, app.json, app.config.ts, eas.json, tsconfig.json, vitest.config.mts	
└─ src/	
└─ app/	LES ÉCRANS (Expo Router)
└─ _layout.tsx	Racine : thème, session, protection des espaces
└─ connexion.tsx	Connexion parent (numéro + code SMS)
└─ connexion-personnel.tsx	Connexion personnel (email + mot de passe)
└─ (parent)/	Espace parent : onglets + écrans de détail
└─ (personnel)/	Espace personnel : scanner, appareil, comportement
└─ components/	Briques d'interface (ui.tsx, entête, choix de l'enfant...)
└─ lib/	Appels API, session, thème, push, stockage, formats, types

Le principe d'organisation : dans l'API, **un dossier = une fonctionnalité**, toujours avec les mêmes fichiers :

Fichier	Contenu
xxx.module.ts	déclare le module
xxx.controller.ts	les routes HTTP
xxx.service.ts	la logique et les accès à la base
xxx.dto.ts	la forme et la validation des données reçues
xxx.regles.ts	les règles métier pures (sans base), faciles à tester
xxx.regles.spec.ts	les tests unitaires de ces règles

A.2 Les fichiers de la racine

package.json (racine)

```
{
  "name": "suivi_eleve", // nom du projet
  "version": "0.1.0", // version (majeure.mineure.correctif)
  "private": true, // jamais publié sur le registre npm
  "description": "Suivi des élèves pour les parents : API, web et mobile",
  "packageManager": "pnpm@12.9.1", // version de pnpm imposée
  "engines": { "node": ">=22" }, // Node.js 22 minimum
  "scripts": {
    "db:up": "docker compose up -d", // démarre PostgreSQL et Redis (-d : en arrière-plan)
    "db:down": "docker compose down", // les arrête
    "db:migrate": "pnpm --filter @suivi-eleve/api db:migrate", // --filter : exécute dans le projet api
    "dev:api": "pnpm --filter @suivi-eleve/api start:dev",
    "build": "pnpm -r build", // -r (récursif) : dans les 3 projets
    "lint": "pnpm -r lint",
    "typecheck": "pnpm -r typecheck",
    "test": "pnpm -r test",
    "format": "prettier --write .", // reformate tout le dépôt
  }
}
```

```

    "format:check": "prettier --check ." // vérifie sans modifier (utilisé par la CI)
  },
  "devDependencies": { "prettier": "^3.9.9" }
}

```

(Le JSON n'accepte pas de commentaires : les `//` ci-dessus sont ajoutés pour l'explication.)

`pnpm-workspace.yaml`

```

packages: # les dossiers qui sont des projets du monorepo
- api
- web
- mobile
allowBuilds: # paquets autorisés à exécuter un script à l'installation
  '@prisma/engines': true # Prisma doit télécharger son moteur
esbuild: true
sharp: false # inutile ici : refusé (sécurité, rapidité)

```

`.prettierrc`, `.editorconfig`, `.gitattributes`

```

{ "singleQuote": true, "trailingComma": "all" }

```

'texte' plutôt que "texte", et une virgule après le dernier élément d'une liste sur plusieurs lignes (les modifications futures ne changent alors qu'une ligne).

`.editorconfig` : `indent_size = 2` (indentation de 2 espaces), `end_of_line = lf` (fin de ligne Linux), `insert_final_newline = true` (ligne vide finale), `trim_trailing_whitespace = true` (pas d'espaces en fin de ligne).

`.gitattributes` : `* text=auto eol=lf` (Git convertit les fins de ligne en LF, même sous Windows) ; `*.png binary` (ne jamais modifier les images).

`docker-compose.yml` (développement)

```

name: suivi_eleve # préfixe des conteneurs (suivi_eleve-postgres-1)
services:
  postgres:
    image: postgres:17-alpine # image officielle, version 17, base Linux Alpine (légère)
    restart: unless-stopped # redémarre seul sauf arrêt volontaire
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-suivi} # variable du .env, « suivi » par défaut
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-suivi_dev}
      POSTGRES_DB: ${POSTGRES_DB:-suivi_eleve}
    ports:
      - '${POSTGRES_PORT:-5432}:5432' # port de l'ordinateur : port du conteneur
    volumes:
      - postgres_data:/var/lib/postgresql/data # données gardées hors du conteneur
    healthcheck: # Docker vérifie que la base répond
      test: ['CMD-SHELL', 'pg_isready -U ... -d ...']
      interval: 5s # toutes les 5 secondes
      retries: 10
  redis:
    image: redis:8-alpine
    ports: ['${REDIS_PORT:-6379}:6379']
    healthcheck: { test: ['CMD', 'redis-cli', 'ping'] } # Redis répond « PONG »

```

```
volumes: # déclaration des volumes nommés
  postgres_data:
  redis_data:
```

Partie B — Mémento de syntaxe TypeScript

Toutes les constructions rencontrées dans le projet, **expliquées une fois ici**. Le reste du document y renvoie (« voir B.x »).

B.1 Importer / exporter.

```
import { Injectable } from '@nestjs/common'; // un élément nommé d'un paquet
import helmet from 'helmet'; // l'élément « par défaut » d'un paquet
import type { Request } from 'express'; // seulement un type (disparaît à la compilation)
import { PAYS } from '../common/pays.js'; // un fichier local (../ = dossier parent)
export const VERSION_CONSETEMENT = '2026-10'; // rend disponible pour les autres fichiers
export default function manifest() { ... } // l'export « par défaut » du fichier
```

Dans l'API, les imports locaux se terminent par `.js` (règle des modules ES de Node.js). Dans le site et le mobile, `@/` désigne la racine du projet (`@/lib/api` = `web/lib/api.ts`).

B.2 Variables. `const x = 1;` (ne sera pas réaffectée — la grande majorité), `let x = 1;` (pourra changer). Jamais `var` (ancienne syntaxe).

B.3 Fonctions fléchées. `const double = (n: number) => n * 2;` est une fonction courte. Avec un bloc : `(n) => { return n * 2; }`. Si on renvoie un objet : `() => ({ a: 1 })` (parenthèses autour des accolades).

B.4 Chaînes de modèle. ``Bonjour ${prenom}`` : entre accents graves, `${...}` insère une valeur.

B.5 Déstructuration. `const { utilisateur } = requete;` = `const utilisateur = requete.utilisateur;`. Pour un tableau : `const [type, jeton] = 'Bearer abc'.split(' ');`. Avec renommage et reste : `const { tuteur, ecole, ...reste } = u;` (`reste` contient toutes les autres propriétés).

B.6 Décomposition (spread). `{ ...a, b: 2 }` copie toutes les propriétés de `a` puis ajoute/remplace `b`. `[...liste, x]` copie un tableau. `f(...roles)` passe chaque élément comme argument.

B.7 Chaînage optionnel et valeur par défaut.

- `a?.b` : lit `b` seulement si `a` existe (sinon `undefined`, pas d'erreur).
- `a ?? b` : `a` si `a` n'est ni `null` ni `undefined`, sinon `b`.
- `a ??= b` : donne la valeur `b` à `a` seulement si `a` est vide.
- `a || b` : `b` si `a` est « faux » (vide, 0, false...). `a && b` : `b` seulement si `a` est vrai (très utilisé en JSX pour afficher sous condition).
- `x!` : « je garantis que `x` n'est pas vide » (à utiliser avec prudence).

B.8 Ternaire. `condition ? siVrai : siFaux`. Imbriqué : `a ? 1 : b ? 2 : 3`.

B.9 Asynchrone. Une opération lente (base, réseau) renvoie une **promesse** (`Promise`). `await` attend son résultat ; une fonction qui utilise `await` est marquée `async`. `Promise.all([p1, p2])` lance plusieurs

opérations **en même temps** et attend toutes. `void f()` lance sans attendre (volontairement). `try { ... }` `catch (e) { ... }` `finally { ... }` attrape les erreurs.

B.10 Types.

- `string`, `number`, `boolean`, `Date`, `null`, `undefined`, `unknown` (inconnu, à vérifier), `any` (n'importe quoi, évité).
- `string[]` : tableau de textes. `string | null` : union (l'un ou l'autre). `'clair' | 'sombre'` : seulement ces deux textes.
- `interface X { a: string; b?: number }` : forme d'un objet ; `?` = facultatif. `type X = ...` : nom donné à un type.
- `Record<Role, string>` : objet dont les clés sont les rôles et les valeurs des textes.
- **Génériques** : `Page<T>` = une page de « n'importe quel type T » ; `lireApi<Eleve[]>(…)` précise T.
- `as const` : fige les valeurs (le tableau devient en lecture seule et ses éléments gardent leur valeur exacte).
- `satisfies Type` : vérifie qu'une valeur respecte un type **sans perdre** son type précis.
- `typeof x` (dans un type) : « le type de la variable x ». `keyof T` : les clés de T. `(typeof eleves)` `[number]` : le type d'un élément du tableau.
- `x as Type` : « traite x comme ce type » (conversion de confiance).
- `Exclude<A, B>` : A sans B. `Partial<T>`, `Pick<T, ...>` : variantes d'un type.

B.11 Classes.

```
export class ErreurApi extends Error {
  // hérite de Error
  constructor(
    readonly statut: number,
    message: string,
  ) {
    super(message); // appelle le constructeur de Error
  }
}
```

`private readonly prisma: PrismaService` dans un constructeur **crée et remplit** automatiquement la propriété `this.prisma` ; `private` = invisible de l'extérieur, `readonly` = non modifiable. `abstract class` = modèle qu'on ne peut pas utiliser directement, seulement hériter. `implements OnModuleInit` = la classe promet d'avoir la méthode `onModuleInit`.

B.12 Décorateurs. `@Quelquechose(...)` posé au-dessus d'une classe, d'une méthode, d'une propriété ou d'un paramètre : il y attache des informations ou un comportement (très utilisé par NestJS et class-validator).

B.13 Tableaux. `.map(f)` transforme chaque élément ; `.filter(f)` garde ceux qui passent le test ; `.find(f)` le premier qui passe ; `.some(f)` « au moins un » ; `.every(f)` « tous » ; `.reduce((somme, x) => somme + x, 0)` cumule ; `.includes(x)` contient ? ; `.slice(0, 3)` les 3 premiers ; `.join(',')` assemble en texte ; `.at(-1)` le dernier ; `.flatMap` transforme et aplatit. `for (const x of liste) { ... }` parcourt.

B.14 Map et Set. `new Map<string, X>()` : dictionnaire (`.get`, `.set`, `.has`, `.values()`). `new Set(liste)` : ensemble sans doublon ; `[...new Set(liste)]` supprime les doublons d'un tableau.

B.15 Expressions régulières. `/^\d{6}$/` = « exactement 6 chiffres » (`^` début, `\d` chiffre, `{6}` six fois, `$` fin). `/[\s.()-]/g` = « espace, point, parenthèse ou tiret, partout » (`g` = global). `.test(texte)` vérifie, `.replace(regex, remplacement)` remplace.

B.16 Objets utiles. `new Date()` maintenant ; `date.toISOString()` → `2026-10-08T09:30:00.000Z` ; `Date.now()` en millisecondes ; `Intl.DateTimeFormat('fr-FR', ...)` formate une date à la française ; `Number(x)` convertit en nombre ; `JSON.stringify(objet)` / `JSON.parse(texte)` ; `encodeURIComponent` protège un texte placé dans une URL. `1_000_000` = un million (le `_` est seulement visuel).

B.17 Commentaires. `//` une ligne, `/*` plusieurs lignes `*/`, `/** documentation */` (affiché par l'éditeur au survol de la fonction).

Partie C — L'API (NestJS)

C.1 Le démarrage

`api/package.json` — ce qui est particulier

- `"type": "module"` : le projet utilise les modules ES (B.1).
- Scripts : `start:dev` = `nest start --watch` (redémarre à chaque modification) ; `start:prod` = `node dist/main` (le code compilé) ; `lint` = `oxlint --type-aware` ; `test` = `vitest run` ; `test:e2e` = tests de bout en bout ; `typecheck` = `tsc --noEmit` ; `postinstall` = `prisma generate` (le client Prisma est régénéré après chaque installation).

`api/tsconfig.json` — options du compilateur

Option	Sens
<code>"module": "nodenext"</code>	produire des modules ES compatibles Node.js
<code>"target": "ES2023"</code>	JavaScript moderne
<code>"strict": true</code>	vérifications de types maximales
<code>"experimentalDecorators", "emitDecoratorMetadata"</code>	autorisent les décorateurs (B.12) et l'injection de dépendances de NestJS
<code>"outDir": "./dist"</code>	le code compilé va dans <code>dist/</code>
<code>"strictPropertyInitialization": false</code>	permet <code>email: string;</code> dans un DTO sans valeur initiale
<code>"types": ["vitest/globals", ...]</code>	<code>describe</code> , <code>it</code> , <code>expect</code> connus sans import dans les tests

`api/src/main.ts` — le point de départ

```
import { NestFactory } from '@nestjs/core'; // la « fabrique » d'applications NestJS
import { AppModule } from './app.module.js'; // le module racine (C.1)
import { configurerApplication } from './app.setup.js';

async function bootstrap() {
  // « amorcer » : démarre tout
  const app = await NestFactory.create(AppModule); // crée l'application à partir du module racine
```

```

configurerApplication(app); // réglages communs (fichier suivant)
// Une ou plusieurs origines séparées par des virgules (site web, version navigateur du mobile).
const origines = (process.env.WEB_ORIGIN ?? 'http://localhost:3001') // process.env : variables
d'environnement
  .split(',') // "a,b" → ["a", "b"]
  .map((o) => o.trim()) // enlève les espaces (B.13)
  .filter(Boolean); // supprime les éléments vides
app.enableCors({ origin: origines }); // CORS : quels sites ont le droit d'appeler l'API depuis un
navigateur
await app.listen(process.env.API_PORT ?? 3100); // écoute sur le port 3100
}
await bootstrap(); // « await » au niveau du fichier (permis en module ES)

```

CORS : par sécurité, un navigateur refuse qu'une page d'un site appelle un autre site, sauf si celui-ci l'autorise. Ici seule l'adresse du site (et de la version navigateur du mobile) est autorisée.

api/src/app.setup.ts — les réglages communs

```

export function configurerApplication(app: INestApplication): void { // void : ne renvoie rien
  const express = app as NestExpressApplication; // accès aux fonctions d'Express (B.10 « as »)
  express.set('trust proxy', 'loopback, uniquelocal'); // croire l'en-tête X-Forwarded-For seulement
  venant // de la machine elle-même ou d'un réseau privé
  (Caddy, le site)
  express.use(helmet({ ... })); // en-têtes de sécurité ; contentSecurityPolicy
  // defaultSrc 'none' : l'API ne sert ni page ni
  script
  express.use(memoriserRequete); // retient l'IP pour le journal d'audit (C.7)
  app.setGlobalPrefix('api'); // toutes les routes commencent par /api
  app.useGlobalPipes( // un « pipe » appliqué à toutes les routes
    new ValidationPipe({
      whitelist: true, // retire les champs non déclarés dans le DTO
      forbidNonWhitelisted: true, // ... et même refuse la requête s'il y en a
      transform: true, // convertit le JSON reçu en objet DTO (types,
valeurs par défaut)
    }),
  );
}

```

Cette fonction est **séparée** de `main.ts` pour que les tests e2e démarrent l'application avec **exactement** les mêmes réglages.

api/src/app.module.ts — l'assemblage

```

@Module({
  // décorateur : cette classe est un module NestJS
  imports: [
    // les modules utilisés
    ConfigModule.forRoot({ isGlobal: true, envFilePath: ['.env', '../.env'] }),
    // ↑ lit le fichier .env (dans api/ ou à la racine) ; isGlobal : ConfigService disponible partout
    PrismaModule,
    AuditModule,
    PlateformeModule,
    SecuriteModule,
    NotificationsModule,
    StockageModule,
    AuthModule,
    ClassesModule,
  ],
})

```

```

    TuteursModule,
    ElevesModule,
    AppareilsModule,
    AnnoncesModule,
    AbsencesModule,
    MatieresModule,
    ResultatsModule,
    ComportementsModule,
    PaiementsModule,
    EvenementsModule,
    UtilisateursModule,
  ],
  controllers: [AppController], // contrôleurs propres à ce module
  providers: [AppService], // services (« fournisseurs ») propres à ce module
})
export class AppModule {} // classe vide : tout est dans le décorateur

```

Ajouter une fonctionnalité = créer son module puis l'ajouter à cette liste.

app.controller.ts et app.service.ts — le plus petit exemple complet

```

@Controller() // contrôleur sans préfixe : ses routes sont /api/...
export class AppController {
  constructor(private readonly appService: AppService) {} // injection de dépendance (B.11)

  @Public() // pas besoin d'être connecté (C.4)
  @Get('sante') // GET /api/sante
  getSante(): EtatSante {
    // ce que renvoie la méthode devient la réponse JSON
    return this.appService.getSante();
  }
}

```

```

export interface EtatSante {
  statut: 'ok';
  service: string;
}
@Injectable() // la classe peut être injectée ailleurs
export class AppService {
  getSante(): EtatSante {
    return { statut: 'ok', service: 'suivi_eleve-api' };
  }
}

```

Cette route sert aux vérifications : le script de mise à jour et Docker l'appellent pour savoir si l'API est vivante.

C.2 La base de données (Prisma)

api/prisma7.config.ts

```

config({ path: ['.env', '../.env'], quiet: true }); // charge le .env (dotenv)
export default defineConfig({
  schema: 'prisma/schema.prisma', // où est le schéma
  migrations: { path: 'prisma/migrations', seed: 'tsx prisma/seed.ts' }, // migrations + commande du seed
  datasource: { url: process.env.DATABASE_URL }, // adresse de la base :
  postgresql://user:mdp@hote:5432/base
});

```

L'en-tête :

```
generator client {
  provider      = "prisma-client"      // génère le client TypeScript
  output        = "../src/generated/prisma" // ... dans ce dossier
  moduleFormat  = "esm"                // en modules ES
  importFileExtension = "js"           // avec des imports en .js (B.1)
}
datasource db { provider = "postgresql" } // type de base
```

Une énumération (liste fermée de valeurs, stockée comme un type PostgreSQL) :

```
enum Role {
  SUPER_ADMIN // concepteur de l'application
  ADMIN
  SECRETARIAT
  ENSEIGNANT
  SURVEILLANT
  COMPTABLE
  PARENT
}
```

Les autres `enum` suivent le même principe : `Pays` (GN, CI, SN), `StatutEleve` (ACTIF, ARCHIVE), `StatutAppareil`, `TypeNotification`, `CanalNotification` (SMS, EMAIL, PUSH, APPLICATION), `StatutNotification` (EN_FILE → ENVOYEE → DELIVREE → LUE, ou ECHOUÉE)...

Un modèle (une table) — `Eleve` :

```
model Eleve {
  id          String      @id @default(uuid(7)) // clé primaire, UUID v7 généré automatiquement
  ecoleId     String      @map("ecole_id")      // champ ecoleId en TS, colonne ecole_id en SQL
  matricule   String      @unique                // pas deux élèves avec le même matricule
  prenom      String      // texte obligatoire
  genre       Genre       // valeur de l'énumération Genre
  dateNaissance DateTime @map("date_naissance") @db.Date // @db.Date : date sans heure
  telephone   String?     // ? = facultatif (peut être null)
  statut      StatutEleve @default(ACTIF)        // valeur par défaut
  classeId    String?     @map("classe_id")
  creeLe      DateTime    @default(now()) @map("cree_le") // date de création automatique
  modifieLe   DateTime    @updatedAt @map("modifie_le")  // mis à jour automatiquement
  creePar     String?     @map("cree_par")         // qui a créé (id utilisateur)

  ecole      Ecole @relation(fields: [ecoleId], references: [id]) // relation : ecoleId pointe vers
  Ecole.id
  classe     Classe? @relation(fields: [classeId], references: [id], onDelete: SetNull)
               // si la classe est supprimée, classeId devient null (au lieu d'une erreur)
  tuteurs   EleveTuteur[] // relation inverse : la liste des liens tuteurs
  ...
  @@index([ecoleId, statut]) // index pour les recherches « élèves actifs de
  l'école »
  @@map("eleves")           // nom de la table SQL
}
```

Conventions du schéma (commentaire en tête du fichier) : camelCase en TypeScript / snake_case en SQL ; chaque table a `cree_le`, `modifie_le`, `cree_par` ; montants en **entiers** ; notes en `Decimal` .

Autres attributs rencontrés :

- `@@unique([ecoleId, contact1])` : unicité sur **deux colonnes ensemble**.
- `@@id([eleveId, tuteurId])` : clé primaire composée (table de liaison `EleveTuteur`).
- `onDelete: Cascade` : supprimer le parent supprime aussi les lignes liées.
- `onDelete: Restrict` : interdit de supprimer une école qui a des utilisateurs.
- `@relation("EnseignantPrincipal", ...)` : relation **nommée**, nécessaire quand deux relations relient les mêmes tables (un utilisateur est auteur ET validateur de comportements).
- `@default(dbgenerated("(CURRENT_DATE + 29)))` : valeur par défaut calculée par PostgreSQL (fin de l'essai gratuit).
- `Json?` : colonne JSON libre (détails du journal d'audit).
- `@db.Decimal(5, 2)` : nombre à 5 chiffres dont 2 après la virgule (moyenne sur 20).
- `///` : commentaire de documentation recopié dans le client généré.

Les 24 modèles par thème :

Thème	Modèles
École et comptes	<code>Ecole</code> , <code>Utilisateur</code> , <code>JettonRafrachissement</code> , <code>CodeOtp</code> , <code>JetonPush</code>
Scolarité	<code>AnneeScolaire</code> , <code>Periode</code> , <code>Classe</code> , <code>HistoriqueClasse</code>
Élèves	<code>Eleve</code> , <code>Tuteur</code> , <code>EleveTuteur</code>
Appareils	<code>Appareil</code> , <code>IncidentAppareil</code>
Résultats	<code>Matiere</code> , <code>Enseignement</code> , <code>MoyenneMatiere</code> , <code>Resultat</code> , <code>DecisionAnnuelle</code>
Vie scolaire	<code>Comportement</code> , <code>Absence</code> , <code>RappelPaiement</code>
Annonces	<code>Annonce</code> , <code>AnnonceClasse</code> , <code>ReponseAnnonce</code>
Messages	<code>Notification</code> , <code>PreferenceNotification</code> , <code>ModeleMessage</code>
Suivi	<code>JournalAudit</code> , <code>PaiementAbonnement</code>

`api/src/prisma/prisma.service.ts`

```
@Injectable()
export class PrismaService extends PrismaClient implements OnModuleDestroy {
  // hérite du client généré : this.eleve.findMany(...), this.annonce.create(...)...
  constructor(config: ConfigService) {
    super({
      // configure le PrismaClient parent
      adapter: new PrismaPg({
        // adaptateur pour le pilote PostgreSQL « pg »
        connectionString: config.getOrThrow<string>('DATABASE_URL'), // erreur au démarrage si absent
      }),
    });
  }
  async onModuleDestroy() {
    // cycle de vie : à l'arrêt de l'application
    await this.$disconnect(); // ferme proprement les connexions
  }
}
```

api/src/prisma/prisma.module.ts

```
@Global() // module global : PrismaService injectable partout sans l'importer
@Module({ providers: [PrismaService], exports: [PrismaService] }) // exports : le rend visible aux autres modules
export class PrismaModule {}
```

Les modules `SecuriteModule`, `NotificationsModule` et `AuditModule` suivent le même modèle `@Global()`.

api/prisma/seed.ts

Script exécuté par `pnpm db:seed`. Il refuse de tourner en production (`if (process.env.NODE_ENV === 'production') throw ...`), crée son propre `PrismaClient`, puis fait des `upsert` (créer ou mettre à jour) sur des identifiants fixes : on peut le relancer sans créer de doublons. Il fabrique une école, 2 classes, 20 élèves avec des prénoms sénégalais, 25 tuteurs, 10 appareils et des comptes du personnel.

Les migrations

Chaque dossier `prisma/migrations/AAAAMMJJHHMMSS_nom/migration.sql` contient du SQL généré, par exemple `CREATE TABLE "eleves" (...), ALTER TABLE ... ADD COLUMN ..., CREATE INDEX ...`. Elles s'appliquent dans l'ordre. Elles racontent l'histoire du projet : `init` → `auth` → `moteur_notifications` → ... → `espace_concepteur_abonnements`.

C.3 Les outils communs (`api/src/common`)

dates.ts

```
/** Date calendaire « AAAA-MM-JJ » (colonnes @db.Date, stockées à minuit UTC). */
export const versJour = (date: Date): string => date.toISOString().slice(0, 10);
// "2026-10-08T09:30:00.000Z".slice(0, 10) → "2026-10-08"

export const depuisJour = (jour: string): Date =>
  new Date(`${jour}T00:00:00.000Z`);

export function calculerAge(
  dateNaissance: Date,
  aujourdHui = new Date(),
): number {
  // aujourdHui = new Date() : paramètre avec valeur par défaut (les tests peuvent fixer la date)
  let age = aujourdHui.getUTCFullYear() - dateNaissance.getUTCFullYear();
  const anniversairePasse =
    aujourdHui.getUTCMonth() > dateNaissance.getUTCMonth() || // mois déjà passé
    (aujourdHui.getUTCMonth() === dateNaissance.getUTCMonth() &&
    aujourdHui.getUTCDate() >= dateNaissance.getUTCDate()); // même mois, jour atteint
  if (!anniversairePasse) age--; // pas encore eu son anniversaire
  return age;
}
```

UTC : temps universel. Les 3 pays du projet sont à l'heure GMT (= UTC) toute l'année, ce qui simplifie les calculs.

pagination.ts

```
export class PaginationDto {
  @IsOptional() // facultatif
  @Type(() => Number) // "2" (texte dans l'URL ?page=2) → 2 (nombre)
  @IsInt()
  @Min(1) // entier ≥ 1
  page: number = 1; // valeur par défaut

  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(100)
  parPage: number = 25;
}

export interface Page<T> {
  elements: T[];
  total: number;
  page: number;
  parPage: number;
  pages: number;
}

export function page<T>(
  elements: T[],
  total: number,
  { page, parPage }: PaginationDto,
): Page<T> {
  return { elements, total, page, parPage, pages: Math.ceil(total / parPage) }; // arrondi supérieur
}

export const sauter = ({ page, parPage }: PaginationDto) => ({
  skip: (page - 1) * parPage, // nombre de lignes à sauter (page 3 de 25 : sauter 50)
  take: parPage, // nombre de lignes à prendre
});
```

Toute liste de l'API renvoie un objet `Page<T>`. Les DTO de filtre « héritent » de `PaginationDto` (`class FiltreAnnoncesDto extends PaginationDto`).

pays.ts

Une table `PAYS: Record<Pays, ReglagesPays>` (B.10) donne pour GN, CI, SN : le nom, la **monnaie** (GNF ou FCFA), l'**indicatif** (224, 225, 221), le nombre de chiffres et un exemple de numéro. Elle est **copiée** dans `web/lib/pays.ts` et `mobile/src/lib/format.ts` (le site et le mobile n'importent pas le code de l'API).

roles.ts

```
export const PERSONNEL = [
  Role.ADMIN,
  Role.SECRETARIAT,
  Role.ENSEIGNANT,
  Role.SURVEILLANT,
  Role.COMPTABLE,
] as const;

export const GESTION_SCOLARITE = [Role.ADMIN, Role.SECRETARIAT] as const;
```

Des groupes de rôles réutilisés dans `@Roles(...PERSONNEL)` (B.6).

validation.ts

```
export const REGEX_TELEPHONE = /^\[1-9\]\d{7,14}$/; // E.164 : « + », un chiffre non nul, puis 7 à 14 chiffres
export const normaliserTelephone = (valeur: string): string =>
  valeur.replace(/\s.()-]/g, '');

export function versE164(valeur: string, indicatif: string): string {
  let numero = normaliserTelephone(valeur);
  if (numero.startsWith('00')) numero = `+${numero.slice(2)}`; // 00224... → +224...
  if (/^\d+$/.test(numero)) {
    // seulement des chiffres (pas de +)
    numero = numero.startsWith(indicatif)
      ? `+${numero}`
      : `+${indicatif}${numero}`;
  }
  return numero;
}

/** Décorateur composé : nettoie puis vérifie un numéro. */
export const EstTelephone = () =>
  applyDecorators(
    // assemble plusieurs décorateurs en un seul
    Transform(({ value }: { value: unknown }) =>
      // class-transformer : modifie la valeur reçue
      typeof value === 'string' ? normaliserTelephone(value) : value,
    ),
    Matches(REGEX_TELEPHONE, {
      message: 'Le numéro doit être au format international...',
    }),
  );

export const Nettoyer = () =>
  Transform(({ value }) => {
    // enlève les espaces autour ; "" → undefined
    if (typeof value !== 'string') return value;
    const nettoye = value.trim();
    return nettoye === '' ? undefined : nettoye;
  });

export const EstEmail = () =>
  applyDecorators(
    Transform(({ value }) =>
      typeof value === 'string' ? value.trim().toLowerCase() : value,
    ),
    IsEmail({}, { message: "L'adresse email n'est pas valide." }),
  );
```

E.164 : le format international des numéros (+224621123456). Règle du projet : tous les numéros sont stockés ainsi.

erreurs-prisma.ts , redis.ts

- `estDoublon(erreur)` : vrai si Prisma a refusé une écriture pour cause de doublon (code `P2002`). Sert à afficher « ce numéro existe déjà » au lieu d'une erreur 500.
- `optionsFile(config)` : l'adresse Redis et le préfixe des clés, communs à toutes les files BullMQ.
`maxRetriesPerRequest: null` est exigé par BullMQ. `traitementActif(config)` : faux si `NOTIFICATIONS_TRAITEMENT=non` (pour faire tourner les travailleurs dans un autre processus).

C.4 L'authentification (`api/src/auth`)

Les décorateurs personnalisés (`auth/decorators`)

```
export const CLE_PUBLIC = 'public';
/** Rend une route accessible sans jeton (toutes les autres routes l'exigent). */
export const Public = () => SetMetadata(CLE_PUBLIC, true);
```

`SetMetadata(clé, valeur)` colle une étiquette sur la route. Les gardes liront cette étiquette avec le `Reflector`. Même principe pour :

- `Roles(...roles)` → étiquette `roles` = liste des rôles autorisés (`...roles` = « paramètre reste » : tous les arguments réunis en tableau) ;
- `OuvertAuConcepteur()` → route qu'un SUPER_ADMIN peut aussi utiliser (son propre profil) ;
- `ParamEleve('id')` → nom du paramètre d'URL qui contient l'id de l'élève.

```
/** Injecte l'utilisateur connecté dans un paramètre de méthode de contrôleur. */
export const UtilisateurCourant = createParamDecorator(
  (_, unknown, ctx: ExecutionContext) =>
    // _ : paramètre volontairement ignoré
    ctx.switchToHttp().getRequest<RequeteAuthentifiee>().utilisateur,
);
```

Dans un contrôleur, `@UtilisateurCourant()` `u: UtilisateurConnecte` donne directement l'utilisateur que `JwtAuthGuard` a placé sur la requête.

`auth/auth.types.ts`

Les formes des données : `UtilisateurConnecte` (id, rôle, école, tuteur), `ChargeJeton` (contenu du JWT : `sub` = identifiant, mot standard des JWT), `RequeteAuthentifiee` (la requête Express **étendue** avec `utilisateur`), `Session` (les deux jetons + l'utilisateur).

`auth/guards/jwt-auth.guard.ts` — garde n°1 : « êtes-vous connecté ? »

```
@Injectable()
export class JwtAuthGuard implements CanActivate {
  // un garde implémente canActivate
  constructor(
    private readonly jwt: JwtService,
    private readonly reflector: Reflector,
  ) {}

  async canActivate(ctx: ExecutionContext): Promise<boolean> {
    // true = la requête passe
    const estPublic = this.reflector.getAllAndOverride<boolean>(CLE_PUBLIC, [
      ctx.getHandler(), // l'étiquette sur la méthode...
      ctx.getClass(), // ... ou sur la classe du contrôleur
    ]);
    if (estPublic) return true; // @Public() : on laisse passer

    const requete = ctx.switchToHttp().getRequest<RequeteAuthentifiee>();
    const [type, jeton] = requete.headers.authorization?.split(' ') ?? []; // "Bearer xxx" →
    ["Bearer", "xxx"]
    if (type !== 'Bearer' || !jeton) {
```

```

    throw new UnauthorizedException("Jeton d'accès manquant."); // → réponse HTTP 401
  }
  let charge: ChargeJeton;
  try {
    charge = await this.jwt.verifyAsync<ChargeJeton>(jeton); // vérifie signature + expiration
  } catch {
    throw new UnauthorizedException("Jeton d'accès invalide ou expiré.");
  }
  requete.utilisateur = {
    id: charge.sub,
    role: charge.role,
    ecoleId: charge.ecoleId,
    tuteurId: charge.tuteurId ?? null,
  }; // pour la suite de la requête
  return true;
}
}

```

Exceptions HTTP de NestJS : lancer `UnauthorizedException` produit une réponse 401 ; `ForbiddenException` 403 ; `NotFoundException` 404 ; `BadRequestException` 400 ; `HttpException`(message, `HttpStatus.TOO_MANY_REQUESTS`) 429. Le message est en français et s'affiche tel quel à l'utilisateur.

`auth/guards/roles.guard.ts` — garde n°2 : « votre rôle le permet-il ? »

```

canActivate(ctx: ExecutionContext): boolean {
  const cibles = [ctx.getHandler(), ctx.getClass()];
  const roles = this.reflector.getAllAndOverride<Role[] | undefined>(CLE_ROLES, cibles);
  const { utilisateur } = ctx.switchToHttp().getRequest<RequeteAuthentifiee>();

  if (utilisateur?.role === Role.SUPER_ADMIN) { // le concepteur :
    const ouvert = this.reflector.getAllAndOverride<boolean>(CLE_OUVERT_AU_CONCEPTEUR, cibles);
    if (ouvert || roles?.includes(Role.SUPER_ADMIN)) return true; // seulement ses routes
    throw new ForbiddenException("L'espace concepteur n'a pas accès aux données des écoles.");
  }
  if (!roles?.length) return true; // pas de @Roles : tout connecté passe
  if (!utilisateur || !roles.includes(utilisateur.role)) {
    throw new ForbiddenException("Votre rôle ne permet pas d'accéder à cette ressource.");
  }
  return true;
}

```

`getAllAndOverride` : si `@Roles` est posé sur la méthode, il **remplace** celui de la classe.

`auth/guards/parent-owns-eleve.guard.ts` — garde n°3 : « est-ce votre enfant ? »

Si l'utilisateur n'est pas PARENT, il passe (le personnel est filtré par école ailleurs). Sinon il lit le nom du paramètre (`@ParamEleve` ou `eleveId` par défaut), puis cherche en base le lien `eleveTuteur` avec la **clé composée** `eleveId_tuteurId`. Pas de lien → 403 « Cet élève n'est pas rattaché à votre compte. » Ce garde n'est pas global : on le pose route par route avec `@UseGuards(ParentOwnsEleveGuard)`.

`auth/auth.module.ts`

```

@Module({
  imports: [
    JwtModule.registerAsync({ // configuration calculée au démarrage
      global: true,
    })
  ]
})

```

```

    inject: [ConfigService], // dépendances de la fabrique
    useFactory: (config: ConfigService) => { // la fabrique
        const secret = config.getOrThrow<string>('JWT_SECRET');
        if (secret.length < 32) throw new Error('JWT_SECRET doit contenir au moins 32 caractères.');
```

if (secret.length < 32) throw new Error('JWT_SECRET doit contenir au moins 32 caractères.');

```

        return { secret, signOptions: { expiresIn: '15m' } }; // jetons d'accès valables 15 minutes
    },
    },
],
controllers: [AuthController],
providers: [
    AuthService, OtpService, ParentOwnsEleveGuard,
    { provide: APP_GUARD, useClass: JwtAuthGuard }, // APP_GUARD : garde appliqué à TOUTES les routes
    { provide: APP_GUARD, useClass: RolesGuard }, // dans cet ordre : connexion puis rôle
],
exports: [ParentOwnsEleveGuard],
})

```

{ provide: X, useClass: Y } : « quand quelqu'un demande X, donne une instance de Y ». C'est la forme longue de l'enregistrement d'un fournisseur.

auth/auth.controller.ts — les routes de connexion

```

@Controller('auth') // routes /api/auth/...
export class AuthController {
    constructor(
        private readonly auth: AuthService,
        private readonly otp: OtpService,
        private readonly limiteur: LimiteurService,
    ) {}

    @Public()
    @Post('connexion') // POST /api/auth/connexion
    @HttpCode(200) // un POST répond 201 par défaut ; ici 200
    async connexion(
        @Body() dto: ConnexionDto,
        @Ip() ip: string,
    ): Promise<Session> {
        // @Body() : le corps JSON, validé selon ConnexionDto ; @Ip() : l'adresse du client
        await this.limiteur.verifier(LIMITES.connexionIp, ip); // anti force brute
        await this.limiteur.verifier(LIMITES.connexionEmail, dto.email);
        return this.auth.connexionPersonnel(dto.email, dto.motDePasse);
    }
    // otp/demande (202 : accepté), otp/verification, rafraichir, deconnexion (204), moi, consentement
}

```

La demande de code répond **toujours** « Si ce numéro est connu, un code a été envoyé » : un pirate ne peut pas savoir quels numéros sont inscrits.

auth/auth.service.ts — les points importants

- `connexionPersonnel` : cherche l'utilisateur par email ; **même si l'email est inconnu**, vérifie le mot de passe contre un hachage factice (`hashFactice`) pour que le temps de réponse ne révèle rien ; refuse si mot de passe faux, compte inactif ou rôle PARENT ; sinon `ouvrirSession` .
- `ouvrirSession` : émet les jetons, note `derniereConnexion` , journalise la connexion (audit).
- `emettreJetons` : refuse si l'école est suspendue (abonnement) ; signe le JWT (`this.jwt.signAsync(charge)`) ; crée un jeton de rafraîchissement aléatoire

(randomBytes(32).toString('base64url')) et n'en stocke que l'**empreinte** (createHash('sha256')).

- **rafraichir** : retrouve le jeton par son empreinte ; s'il était déjà révoqué → **ferme toutes les sessions** (vol probable) ; s'il est expiré → 401 ; sinon le révoque avec `updateMany({ where: { id, revoqueLe: null } })` : si deux appels arrivent en même temps, **un seul** réussit (`count === 0` pour l'autre) — c'est une technique classique contre les « accès concurrents ».
- **profil** : renvoie l'utilisateur, son école (pays, monnaie, indicatif) et, pour un parent, l'état du **consentement** (`accepte: tuteur.consentementVersion === VERSION_CONSENTEMENT`).
- **consentir** : enregistre la version acceptée et la date.

auth/otp.service.ts — la connexion des parents par SMS

```
async demanderCode(telephone: string): Promise<void> {
  const demandesRecentes = await this.prisma.codeOtp.count({
    where: { telephone, creeLe: { gte: new Date(Date.now() - FENETRE_DEMANDES_OTP_MS) } },
  });
  // gte = « supérieur ou égal »
  if (demandesRecentes >= DEMANDES_MAX_OTP) throw new HttpException('Trop de demandes...', 429);

  const tuteur = await this.trouverTuteur(telephone);
  if (!tuteur || (await this.etatEcoles.estSuspendue(tuteur.ecoleId))) return; // silence (même réponse)

  const code = randomInt(0, 1_000_000).toString().padStart(6, '0'); // 0 à 999999, sur 6 chiffres :
  "004217"
  const codeHash = await hash(code); // argon2 : le code n'est jamais
  stocké en clair
  await this.prisma.$transaction([ // les deux opérations ensemble
    this.prisma.codeOtp.updateMany({ where: { telephone, consommeLe: null },
      data: { consommeLe: new Date() } }), // annule les anciens codes
    this.prisma.codeOtp.create({ data: { telephone, codeHash,
      expireLe: new Date(Date.now() + DUREE_VALIDITE_OTP_MS) } }),
  ]);
  await this.sms.envoyer(telephone, `Suivi_eleve : votre code de connexion est ${code}. ...`);
}
```

verifierCode : prend le dernier code non consommé ; **compte l'essai avant de vérifier** (3 essais maximum, même avec des appels simultanés) ; compare avec `verify(hash, code)` ; à la première connexion, **crée le compte parent** (`utilisateur.create` avec `tuteur: { connect: { id } }` = relier à un tuteur existant) ; ouvre la session. `trouverTuteur` cherche d'abord `contact1`, puis `contact2` seulement s'il désigne un seul tuteur.

auth/consentement.ts

Le texte d'information montré au parent à sa première connexion (`TEXTE_CONSENTEMENT`, tableau de paragraphes `as const`) et sa version (`'2026-10'`). Changer le texte → changer la version → tous les parents doivent l'accepter de nouveau.

auth/dto/auth.dto.ts

```
export class VerificationOtpDto extends DemandeOtpDto {
  // hérite du champ telephone
  @Matches(/^d{6}$/, { message: 'Le code contient 6 chiffres.' })
  code: string;
}
```

Un DTO est une **classe** (pas une interface) car les décorateurs de validation doivent exister à l'exécution.

C.5 Un module complet : les annonces ([api/src/annonces](#))

C'est le meilleur exemple pour comprendre **tout le chemin** d'une fonctionnalité. Les autres modules suivent le même plan.

annonces.module.ts

```
@Module({
  imports: [ClassesModule], // a besoin de ClassesService (vérifier qu'une classe est à l'école)
  controllers: [AnnoncesController],
  providers: [AnnoncesService],
  exports: [AnnoncesService], // EvenementsModule le réutilise
})
export class AnnoncesModule {}
```

annonces.dto.ts — les données acceptées

```
const TYPES = [TypeAnnonce.PAS_DE_COURS, TypeAnnonce.LIBERATION_ANTICIPEE];

export class CreerAnnonceDto {
  @IsIn(TYPES, { message: 'Type invalide : PAS_DE_COURS ou LIBERATION_ANTICIPEE.' }) // une valeur de la
  liste
  type: TypeAnnonce;

  @IsEnum(CibleAnnonce, { message: 'Cible invalide : ECOLE ou CLASSES.' }) // une valeur de l'enum
  cible: CibleAnnonce;

  @ValidateIf((a: CreerAnnonceDto) => a.cible === CibleAnnonce.CLASSES) // règle conditionnelle :
  @IsArray() // vérifié seulement si
  cible = CLASSES
  @ArrayMinSize(1, { message: 'Choisissez au moins une classe.' })
  @ArrayMaxSize(100)
  @IsUUID('all', { each: true }) // chaque élément est un
  UUID
  classeIds?: string[];

  @Matches(/^\\d{4}-\\d{2}-\\d{2}$/) { message: 'La date doit être au format AAAA-MM-JJ.' })
  date: string;

  @ValidateIf((a) => a.type === TypeAnnonce.LIBERATION_ANTICIPEE)
  @Matches(/^([01]\\d|2[0-3]):[0-5]\\d$/, { message: "L'heure de sortie doit être au format HH:MM." })
  heure?: string; // 00:00 à 23:59 : ([01]\\d|2[0-3]) = 00-19 ou 20-23

  @ValidateIf((a) => a.type === TypeAnnonce.PAS_DE_COURS)
  @Nettoyer() @IsString({ message: 'Précisez le créneau concerné.' }) @MaxLength(80)
  creneau?: string;

  @IsEnum(MotifAnnonce, { message: 'Le motif est obligatoire.' }) motif: MotifAnnonce;
  @ValidateIf((a) => a.motif === MotifAnnonce.AUTRE) @Nettoyer() @IsString(...) @MaxLength(120)
  motifDetail?: string;
  @IsOptional() @Nettoyer() @IsString() @MaxLength(1000) message?: string;
  @IsOptional() @IsISO8601({}, { message: "Date d'envoi invalide." }) programmeLe?: string;
}

export class FiltreAnnoncesDto extends PaginationDto { // filtres de la liste + pagination
  @IsOptional() @IsIn(TYPES) type?: TypeAnnonce;
```

```
@IsOptional() @IsEnum(StatutAnnonce) statut?: StatutAnnonce;
}
```

Si une règle échoue, le `ValidationPipe` répond **400** avec la liste des messages, avant même d'appeler le contrôleur.

annonces.regles.ts — les règles pures (sans base de données)

```
export const FUSEAU = 'Africa/Dakar'; // heure GMT, sans heure d'été

export const LIBELLES_MOTIF: Record<MotifAnnonce, string> = { // texte lu par les familles
  GREVE: 'grève', COUPURE_ELECTRICITE: "coupure d'électricité", INTEMPERIES: 'intempéries',
  ABSENCE_ENSEIGNANT: "absence d'enseignant", AUTRE: 'autre motif',
};

export function motifTexte(motif: MotifAnnonce, detail?: string | null): string {
  if (motif === MotifAnnonce.AUTRE) return detail?.trim() || LIBELLES_MOTIF.AUTRE; // « Autre » →
  précision saisie
  return LIBELLES_MOTIF[motif];
}

export function instantDakar(jour: string, heure = '00:00'): Date {
  return new Date(`${jour}T${heure}:00.000Z`); // "2026-10-07" + "11:30" → 2026-10-07T11:30Z
}

const formatHeure = new Intl.DateTimeFormat('fr-FR', { timeZone: FUSEAU, hour: '2-digit', minute: '2-digit' });
export const heureFr = (date: Date) => formatHeure.format(date).replace(':', 'h'); // « 11h30 »

export function titreAnnonce(type: TypeAnnonce, dateDebut: Date, classes: string[] | null): string {
  const qui = classes?.length ? classes.join(', ') : 'toute l'école';
  return type === TypeAnnonce.LIBERATION_ANTICIPEE
    ? `Libération anticipée à ${heureFr(dateDebut)} - ${qui}`
    : `Pas de cours le ${jourFr(dateDebut)} - ${qui}`;
}

export function variablesAnnonce(annonce: { ... }): Record<string, string> { // remplissent {date},
{heure}...
  return { date: jourFr(annonce.dateDebut), heure: heureFr(annonce.dateDebut),
    creneau: annonce.creneau ?? '', motif: ..., details: annonce.message ? `\n\n${annonce.message}`
: '' };
}

export const AVANCE_MIN_MS = 60 * 1000; // une minute, en millisecondes
export function erreurProgrammation(programmeeLe: Date, maintenant = new Date()): string | null {
  if (Number.isNaN(programmeeLe.getTime())) return "Date d'envoi invalide."; // date illisible
  if (programmeLe.getTime() < maintenant.getTime() + AVANCE_MIN_MS) return "...au moins une minute...";
  if (programmeLe.getTime() > maintenant.getTime() + 60 * 24 * 3600 * 1000) return "...plus de 60 jours.";
  return null; // null = pas d'erreur
}
```

Pourquoi séparer les règles ? Une fonction pure (même entrée → même sortie, sans base) se teste en une ligne, sans démarrer l'application.

annonces.regles.spec.ts — les tests unitaires

```
describe('annonces', () => {
  // groupe de tests
```

```

const sortie = instantDakar('2026-10-07', '11:30');
it("compose l'instant à l'heure de Dakar et le formate", () => {
  // un test
  expect(sortie.toISOString()).toBe('2026-10-07T11:30:00.000Z'); // « on attend que ... soit ... »
  expect(heureFr(sortie)).toBe('11h30');
});
it('contrôle la date d'un envoi programmé', () => {
  const maintenant = new Date('2026-10-06T10:00:00Z'); // date fixée : le test donne toujours le même
  résultat
  expect(
    erreurProgrammation(new Date('2026-10-06T18:00:00Z'), maintenant),
  ).toBeNull();
  expect(
    erreurProgrammation(new Date('2026-10-06T10:00:30Z'), maintenant),
  ).toMatch(/une minute/);
});
});

```

`toBe` (égalité exacte), `toEqual` (même contenu d'objet), `toBeNull`, `toMatch(regex)`. `it.each([...])` (dans `validation.spec.ts`) répète un test pour chaque ligne d'un tableau.

annonces.controller.ts — les routes

```

@Controller('annonces') // /api/annonces
@Roles(...PERSONNEL) // par défaut : tout le personnel (B.6 : spread du tableau)
export class AnnoncesController {
  constructor(private readonly annonces: AnnoncesService) {}

  @Get() // GET /api/annonces?page=1&type=...
  lister(
    @UtilisateurCourant() u: UtilisateurConnecte,
    @Query() filtre: FiltreAnnoncesDto,
  ) {
    return this.annonces.lister(u, filtre); // @Query() : les paramètres après « ? », validés
  }

  @Get('/:id') // :id = partie variable de l'adresse
  detail(
    @UtilisateurCourant() u: UtilisateurConnecte,
    @Param('id', ParseUUIDPipe) id: string,
  ) {
    return this.annonces.detail(u, id); // ParseUUIDPipe : 400 si id n'est pas un UUID
  }

  @Post()
  @Roles(...GESTION_SCOLARITE) // création : direction et secrétariat seulement (remplace le @Roles de la
  classe)
  creer(
    @UtilisateurCourant() u: UtilisateurConnecte,
    @Body() dto: CreerAnnonceDto,
  ) {
    return this.annonces.creer(u, dto);
  }
  // POST :id/annuler et POST :id/envoyer : même forme, @HttpCode(200)
}

```

Un contrôleur reste **mince** : il ne fait que relier la requête au service.

La sélection réutilisable et son type :

```
const selectionAnnonce = {
  id: true, type: true, titre: true, ...,
  auteur: { select: { prenom: true, nom: true } }, // jointure : l'auteur
  classes: { select: { classe: { select: { id: true, nom: true } } } }, // jointure à travers
  AnnonceClasse
} satisfies Prisma.AnnonceSelect; // B.10 satisfies

type AnnonceBrute = Prisma.AnnonceGetPayload<{ select: typeof selectionAnnonce }>;
// le type EXACT de ce que Prisma renvoie avec cette sélection

const formater = ({ classes, ...a }: AnnonceBrute) => ({ ...a, classes: classes.map((c) => c.classe) });
// aplatit [{classe:{id,nom}}] en [{id,nom}]
```

Le cycle de vie et la file BullMQ :

```
@Injectable()
export class AnnoncesService implements OnModuleInit, OnModuleDestroy {
  private readonly logger = new Logger(AnnoncesService.name); // journal (logs) préfixé du nom de la
  classe
  private file!: Queue; // ! : sera rempli dans onModuleInit
  private travailleur?: Worker;

  async onModuleInit() { // au démarrage de l'API
    this.file = new Queue('annonces', {
      ...optionsFile(this.config),
      defaultJobOptions: {
        attempts: 3, // 3 essais
        backoff: { type: 'exponential', delay: 5_000 }, // 5 s, 10 s, 20 s
        removeOnComplete: 500, removeOnFail: 1000, // garder un historique limité dans Redis
      },
    });
  };
  if (!traitementActif(this.config)) return;
  this.travailleur = new Worker<{ annonceId: string }>(
    'annonces',
    (job) => job.name === 'rappel' ? this.envoyerRappel(job.data.annonceId)
      : this.envoyer(job.data.annonceId), // ce que fait chaque tâche
    optionsFile(this.config),
  );
  this.travailleur.on('error', (e) => this.logger.error(`File annonces : ${e.message}`));
  // Reprogramme ce qui attendait (Redis vidé, redémarrage...)
  const enAttente = await this.prisma.annonce.findMany({ where: { statut: StatutAnnonce.PROGRAMMEE }, ...
});
  for (const a of enAttente) await this.programmer(a.id, a.programmeeLe ?? new Date());
  ...
}

private async programmer(id: string, quand: Date) {
  await this.file.add('envoi', { annonceId: id },
    { jobId: id, // identifiant = id de l'annonce : jamais
      delay: Math.max(0, quand.getTime() - Date.now()) }); // attendre jusqu'à l'heure prévue
}
```

La **base de données reste la vérité** : au redémarrage, on relit les annonces « programmées » et on les remet en file. Redis peut être vidé sans perte.

La création :

```
async creer(u: UtilisateurConnecte, dto: CreerAnnonceDto) {
  const parClasses = dto.cible === CibleAnnonce.CLASSES;
  const classeIds = parClasses ? [...new Set(dto.classeIds ?? [])] : []; // sans doublons (B.14)
  for (const id of classeIds) await this.classes.verifierClasse(u.ecoleId, id); // chaque classe est
  bien à cette école
  const nomsClasses = parClasses ? (await this.prisma.classe.findMany({ ... })).map((c) => c.nom) : null;

  const liberation = dto.type === TypeAnnonce.LIBERATION_ANTICIPEE;
  const dateDebut = instantDakar(dto.date, liberation ? dto.heure : undefined);
  if (Number.isNaN(dateDebut.getTime())) throw new BadRequestException('Date invalide.');
```

const programmeLe = dto.programmeLe ? new Date(dto.programmeLe) : null;
if (programmeLe) { const erreur = erreurProgrammation(programmeLe); if (erreur) throw new
BadRequestException(erreur); }

```
const annonce = await this.prisma.annonce.create({
  data: {
    ecoleId: u.ecoleId, // TOUJOURS l'école de l'utilisateur
(cloisonnement)
    type: dto.type, titre: titreAnnonce(dto.type, dateDebut, nomsClasses), ...,
    statut: programmeLe ? StatutAnnonce.PROGRAMMEE : StatutAnnonce.BROUILLON,
    auteurId: u.id, creePar: u.id,
    classes: { create: classeIds.map((classeId) => ({ classeId, creePar: u.id })), // création
imbriquée
  },
  select: { id: true },
});
await this.audit.journaliser(u, ActionAudit.CREATION, 'Annonce', annonce.id, { type: dto.type, ... });
await this.publier(annonce.id, programmeLe); // envoi tout de suite ou programmé
return this.detail(u, annonce.id); // renvoie l'annonce complète
}
```

L'envoi :

```
async envoyer(annonceId: string): Promise<void> {
  const annonce = await this.prisma.annonce.findUnique({ where: { id: annonceId }, include: { classes: ...
} });
  if (!annonce || (annonce.statut !== BROUILLON && annonce.statut !== PROGRAMMEE)) return; // déjà
envoyée/annulée
  await this.notifications.notifier({ // LE moteur de notifications (C.6)
    ecoleId: annonce.ecoleId,
    type: TYPE_NOTIFICATION[annonce.type],
    cible: annonce.cible === CibleAnnonce.ECOLE ? { ecole: true }
      : { classeIds: annonce.classes.map((c) => c.classeId) },
    variables: ... variablesAnnonce(annonce),
    sourceType: 'annonce', sourceId: annonce.id, // pour retrouver « qui a lu » ensuite
    cleDeduplication: `annonce:${annonce.id}`, // si rejoué, aucun tuteur ne reçoit deux
fois
  });
  await this.prisma.annonce.update({ where: { id: annonce.id },
    data: { statut: StatutAnnonce.ENVOYEE, envoyeeLe: new Date() } });
}
```

Une opération qui peut être **rejouée sans effet de bord** est dite **idempotente** : c'est indispensable avec des files qui réessaient.

Le suivi (`lister`, `detail`, `suivi`) : `prisma.notification.groupBy({ by: ['canal', 'statut'], _count: { _all: true } })` compte les envois par canal et par statut (comme `GROUP BY` en SQL) ; la liste des familles qui n'ont pas lu vient des lignes `APPLICATION` sans `luLe` . `prisma.$transaction([findMany, count])` lit la page et le total en une fois.

Annuler / envoyer maintenant : `exigerProgrammee` vérifie que l'annonce est de cette école et encore « programmée », puis `this.file.remove(id)` retire la tâche différée.

C.6 Le moteur de notifications (`api/src/notifications`)

Le cœur de l'application : **tous** les messages aux familles passent par ici.

`notifications.regles.ts`

```
const { SMS, EMAIL, PUSH } = CanalNotification; // déstructuration d'un objet enum

interface RegleType { canaux: CanalNotification[]; priorite: PrioriteNotification; obligatoire: boolean; }

export const REGLES_TYPE: Record<TypeNotification, RegleType> = {
  LIBERATION_ANTICIPEE: { canaux: [SMS, EMAIL, PUSH], priorite: 'URGENTE', obligatoire: true },
  PAS_DE_COURS: { canaux: [SMS, EMAIL, PUSH], priorite: 'HAUTE', obligatoire: true },
  ABSENCE: { ..., priorite: 'HAUTE', obligatoire: true },
  RAPPEL_PAIEMENT: { ..., priorite: 'NORMALE', obligatoire: false }, // le parent peut le couper
  RECU_PAIEMENT: { canaux: [EMAIL, PUSH], priorite: 'BASSE', obligatoire: false },
  ...
};
export const PRIORITE_FILE = { URGENTE: 1, HAUTE: 2, NORMALE: 3, BASSE: 4 }; // 1 = servi en premier
```

Le tableau de bord des règles : pour chaque type de message, ses canaux, sa priorité et s'il est **obligatoire** (envoyé même si le parent l'a désactivé).

```
/** Remplace {variable} ; une variable inconnue devient une chaîne vide. */
export function rendre(
  modele: string,
  variables: Record<string, string | undefined>,
): string {
  return modele
    .replace(/\{([a-z_]+)\}/g, (_, nom: string) => variables[nom] ?? '') // {heure} → "11h30"
    .replace(/\(\\s*\)/g, '') // supprime les parenthèses restées vides « () »
    .replace(/\t{2,}/g, ' ') // espaces doublés → un seul
    .replace(/ +([.,])/g, '$1') // pas d'espace avant un point ou une virgule ($1 = ce qui a été capturé)
    .trim();
}
```

`versGsm7(texte)` : découpe le texte en caractères affichés (`Intl.Segmenter`), garde ceux de l'alphabet GSM, remplace les autres (`ê` → `e`, `'` → `'`, ... → ...) ou les remplace par `?` . `preparerSms` : une seule ligne, 160 caractères maximum (sinon coupé avec ...). `enumerer(['Awa', 'Ali', 'Fatou'])` → « Awa, Ali et Fatou ». `doitMettreAJour` empêche un statut de revenir en arrière (un accusé « délivré » tardif n'efface pas « lu »).

`modeles.default.ts` et `modeles.service.ts`

Les textes par défaut de chaque message, pour chaque canal :

```
LIBERATION_ANTICIPEE: {
  SMS: { contenu: 'Suivi_eleve : les élèves de {classe} sont libérés à {heure} ({motif}). Merci de
prendre vos dispositions.' },
  EMAIL: { sujet: 'Libération anticipée - {classe}', contenu: `${salutation}Les élèves de {classe} ...
{details}${signature}` },
  PUSH: { ... },
},
```

L'école peut les **remplacer** (table `modeles_message`, page « Notifications » Modèles ») ; `ModelesService` liste, enregistre (en vérifiant les variables) et rétablit les textes.

`notifications.service.ts` — la fonction `notifier`

C'est **la** fonction que tous les modules appellent. Étapes :

1. **Règle du type** : canaux, priorité, obligatoire (`REGLES_TYPE[demande.type]`).
2. **École suspendue ?** Rien ne part.
3. **Élèves visés** : précis (`eleveIds`), classes (`classeIds`) ou toute l'école (`ecole: true`), avec leurs tuteurs, jetons push et préférences — **une seule requête** Prisma avec des `select` imbriqués.
4. **Regroupement par tuteur** : une `Map` tuteur → enfants. Une mère de 3 enfants reçoit **un seul** message qui cite « Awa, Ali et Fatou ».
5. **Déduplication** : on retire les tuteurs qui ont déjà reçu ce message (`cleDeduplication`).
6. **Modèles** de l'école (ou par défaut), dans la **langue** du tuteur sinon en français.
7. **Plafond SMS** mensuel éventuel (jamais pour un message obligatoire).
8. **Construction des lignes** — pour chaque tuteur, avec un `lotId` commun :
 - une ligne **APPLICATION** (historique visible par le parent), toujours, déjà « DÉLIVRÉE » ;
 - une ligne **SMS** vers `contact1` si le canal est prévu et accepté ;
 - une ligne **EMAIL** si le tuteur a un email ;
 - une ligne **PUSH** par appareil enregistré.

```
const accepte = (canal: 'sms' | 'email' | 'push') =>
  regle.obligatoire || !preference || preference[canal]; // obligatoire OU pas de préférence OU
préférence = oui
```

9. **Écriture** de toutes les lignes en une fois (`createManyAndReturn`) puis **mise en file** de celles à envoyer (`this.envoi.ajouter`).

Les autres méthodes servent la consultation : `journal` (liste filtrée pour l'administration), `statistiques` (bilan mensuel, coût SMS), `mesNotifications` (historique du parent + nombre de non lus), `marquerLue` (accusé de lecture : toute la ligne APPLICATION du lot passe à LUE), `preferences` / `enregistrerPreferences` (`upsert` par type, jamais pour un type obligatoire).

`envoi.service.ts` — les files et les essais

- `onModuleInit` : crée **une file par canal** (`notifications-sms`, `-email`, `-push`) avec `attempts` et `backoff` exponentiel, et un **travailleur** par file (`concurrency: 5` : 5 envois en parallèle) ; puis `reprenreEnAttente()` remet en file les notifications restées `EN_FILE` (moins de 7 jours).
- `ajouter(notifications)` : `file.add('envoi', { notificationId }, { jobId: n.id, priority: PRIORITE_FILE[n.priorite] })`.

- `traiter(id, dernierEssai) :`

```
try {
    const resultat = await this.envoyer(n); // appelle le bon fournisseur
    await this.prisma.notification.update({ ... statut: ENVOYEE, fournisseur, referenceFournisseur,
    cout,
    essais: { increment: 1 } ... }); // increment : +1
    en base
} catch (e) {
    const definitive = e instanceof ErreurFournisseur && e.definitive; // ex. numéro invalide :
    inutile de réessayer
    const abandon = definitive || dernierEssai;
    await this.prisma.notification.update({ ... statut: abandon ? ECHOUUE : EN_FILE, erreur: ... });
    if (e instanceof JetonPushInvalide) await this.prisma.jetonPush.deleteMany(...); // appareil
    désinstallé
    if (abandon) await this.basculerSurContact2(n.id); // SMS échoué → même message vers
    contact2
    throw definitive ? new UnrecoverableError(message) : e; // relancer l'erreur : BullMQ réessaiera
    (ou non)
}
```

- `envoyerPush` : distingue un jeton **mobile** (FCM) d'un abonnement **Web Push** (plateforme WEB) et ajoute l'adresse du message à ouvrir au clic.
- `renvoyer` (bouton « Renvoyer » du journal) et `accuserReception` (webhook du fournisseur : DÉLIVRÉE ou ÉCHOUÉE).

`switch (n.canal) { case ...: return ...; default: ... } :` choisit un traitement selon une valeur.

`canaux.service.ts` et `fournisseurs/fournisseurs.ts`

`CanauxService` lit la configuration et **fabrique** les fournisseurs (`creerSms('orange')` → `new OrangeSms({...})`). Il refuse en production les faux fournisseurs, sauf `console` quand `ENVOIS_SIMULES=oui` (phase d'essai actuelle). `envoyerSms` essaie le fournisseur principal puis celui de **secours**.

`fournisseurs.ts` définit des **interfaces** communes (`FournisseurSms` , `FournisseurEmail` , `FournisseurPush` : un nom et une méthode `envoyer`) et une classe par fournisseur qui les **implémente** (`class OrangeSms implements FournisseurSms`). Chaque classe appelle l'API HTTP du fournisseur avec `fetch` et transforme les réponses en `ResultatEnvoi` ou en `ErreurFournisseur` (`definitive` si un 4xx montre que réessayer est inutile). `FournisseurConsole` écrit le message dans les logs ; `FournisseurSimule` sert aux tests. C'est le **patron « stratégie »** : le reste du code ne sait pas quel fournisseur est utilisé.

`notifications.module.ts`

```
@Injectable()
class SmsDirect extends SmsSender { // implémentation concrète de l'interface
    abstraite
    constructor(private readonly canaux: CanauxService) { super(); }
    async envoyer(telephone: string, message: string): Promise<void> {
        await this.canaux.envoyerSms(telephone, preparerSms(message), 'otp'); // sans file : le code doit
        partir tout de suite
    }
}
@Global()
@Module({ controllers: [NotificationsController],
    providers: [CanauxService, EnvoiService, NotificationsService, ModelesService,
```

```

        { provide: SmsSender, useClass: SmsDirect }], // qui demande SmsSender reçoit
SmsDirect
    exports: [NotificationsService, SmsSender] })

```

`api/src/sms/sms.sender.ts` ne contient que la classe abstraite `SmsSender` : `OtpService` dépend de l'idée d'envoyer un SMS, pas d'un fournisseur précis.

`notifications.controller.ts`

Trois groupes de routes : **administration** (journal, statistiques, renvoyer, modèles), **parents** (`mes`, `:id/lue`, `preferences`, `push-web`, `jetons-push`) et **webhooks** des fournisseurs (`@Public()` mais protégés par le secret `WEBHOOK_SECRET` passé dans l'adresse).

C.7 Les services transverses

`securite/contexte-requete.ts`

```

const contexte = new AsyncLocalStorage<{ ip?: string }>(); // « mémoire » propre à chaque requête
export const ipRequete = () => contexte.getStore()?.ip; // lisible partout pendant la requête
export function memoriserRequete(
    req: Request,
    _res: Response,
    next: NextFunction,
) {
    contexte.run({ ip: req.ip }, next); // middleware Express : exécute la suite dans ce contexte
}

```

Un **middleware** est une fonction appelée pour chaque requête avant les routes ; `next()` passe à la suite.

`securite/limiteur.service.ts`

```

async verifier(limite: Limite, identifiant: string): Promise<void> {
    const cle = `${this.prefixe}:limite:${limite.nom}:${identifiant.toLowerCase()}`; // ex.
    suivi:limite:connexion-ip:1.2.3.4
    const [[, compte], [, ttl]] = (await this.redis.multi() // multi : plusieurs commandes Redis d'un
    bloc
        .incr(cle) // +1 au compteur (créé à 1 s'il n'existe
    pas)
        .ttl(cle) // durée de vie restante
        .exec()) as [[null, number], [null, number]];
    if (ttl < 0) await this.redis.expire(cle, limite.fenetreS); // nouveau compteur : expire après la
    fenêtre
    if (compte > limite.max) throw new HttpException({ ... message: limite.message, reessayerDansS: ... },
    429);
}

```

Algorithme de **fenêtre fixe** : au plus N essais par période.

`audit/audit.service.ts`

`journaliser(utilisateur, action, entite, entiteId?, details?, ecoleId?)` ajoute une ligne à `journal_audit` avec l'IP (`ipRequete()`). La direction la consulte dans la page « Journal » (`audit.controller.ts`, réservé à ADMIN ; les actions du concepteur n'y apparaissent pas).

stockage/stockage.service.ts

- `typeImage(contenu)` reconnaît un JPEG, PNG ou WebP à ses **premiers octets** (« nombres magiques » : `FF D8 FF` pour un JPEG), pas à son nom : impossible de déguiser un programme en photo.
- `enregistrer`, `lire`, `supprimer` écrivent dans `STOCKAGE_DIR` (`node:fs/promises`).
- `chemin(cle)` refuse toute clé qui sortirait du dossier (protection contre `../../etc/passwd`).

plateforme/abonnements.regles.ts et etat-ecoles.service.ts

```
export const TARIFS_GNF = { MENSUEL: 150_000, ANNUEL: 1_500_000 }; // hors taxes
export const JOURS_ESSAI = 30; export const JOURS_AVERTISSEMENT = 7; export const JOURS_GRACE = 15;

export function situationAbonnement(ecole, aujourdHui = new Date()): SituationAbonnement {
  const joursRestants = joursEntre(aujourdHui, ecole.finAbonnement);
  const etat =
    ecole.suspendueLe || joursRestants < -JOURS_GRACE ? 'SUSPENDUE' // suspendue à la main ou grâce
    dépassée
    : joursRestants < 0 ? 'EN_RETARD' // dans les 15 jours de grâce
    : joursRestants < JOURS_AVERTISSEMENT ? 'A_RENOUVELER' // 7 derniers jours
    : !ecole.aPaye ? 'ESSAI' : 'ACTIF';
  ...
}
```

`ajouterMois` gère le 31 janvier + 1 mois = 28/29 février. `periodePayee` fait suivre la nouvelle période à la précédente (ou repart du jour du paiement si l'école était suspendue).

`EtatEcolesService.estSuspendue(ecoleId)` est appelé par la connexion, la demande de code SMS et notifier .

cli/initialiser.ts et cli/cles-vapid.ts

Scripts lancés **à la main** sur le serveur (`node dist/cli/initialiser.js --prenoms ... --nom ... --email ...`). `parseArgs` lit les options ; le script crée le compte `SUPER_ADMIN` (ou lui donne un nouveau mot de passe provisoire s'il existe) et ferme ses sessions. `function arreter(message): never` : le type `never` signifie « cette fonction ne revient jamais » (elle quitte le programme). `cles-vapid.ts` génère la paire de clés Web Push.

C.8 Les autres modules (mêmes motifs, particularités)

Module	Routes principales	Ce qu'il a de particulier
classes	GET/POST/PATCH /classes	<code>anneeActive(ecoleId)</code> ; <code>verifierClasse</code> réutilisé partout pour vérifier qu'une classe appartient à l'école
tuteurs	GET/POST/PATCH /tuteurs	un <code>contact1</code> déjà connu désigne le même tuteur (frères et sœurs)

Module	Routes principales	Ce qu'il a de particulier
eleves	/eleves , export , import , :id/changer-classe , archiver , restaurer , tuteurs , donnees , effacer	matricule SE-AAAA-NNNN (eleves.regles.ts , avec nouvel essai si deux inscriptions prennent le même) ; jamais de suppression : archivage ; import-export.ts lit CSV/Excel (détection du séparateur, dates JJ/MM/AAAA, numéros convertis en E.164) et rapporte les erreurs ligne par ligne ; donnees.service.ts exporte toutes les données (droit d'accès) et anonymise un élève archivé (droit à l'effacement)
appareils	/appareils , qr/:code , etiquettes (PDF) , :id/photo , :id/signalements	REGLES_INCIDENT : quel rôle peut signaler perdu / trouvé / confisqué / restitué / usage en classe, depuis quel statut ; contrôle de l' IMEI par la clé de Luhn ; seuilAtteint crée un comportement après N usages en classe dans le mois ; etiquettes.ts dessine 12 QR codes par page A4 sans le nom de l'élève
absences	GET/POST /absences , justifier , justification-parent , DELETE	l'appel enregistre plusieurs absents d'un coup ; une absence non justifiée déclenche notifier(ABSENCE) ; contrainte unique élève + date + créneau (pas de doublon)
matieres	/matieres , classes/:id/enseignements , personnel	qui enseigne quelle matière dans quelle classe = qui saisit quelle moyenne
resultats	periodes , mes-saisies , saisie , moyennes , generaux , publier , decisions , eleves/:eleveId , bulletins/:periodeId	aucun calcul : chaque professeur saisit la moyenne de sa matière, le professeur principal la moyenne générale et le rang ; la publication prévient les familles ; bulletin.ts dessine le bulletin avec pdfkit (doc.text , doc.rect ...) ; un parent ne voit que les résultats publiés
comportements	GET/POST , :id/valider , :id/rejeter , DELETE	gravité 1 à 3 ; un cas grave (3) signalé par un enseignant attend la validation de la direction avant tout message (statutInitial)
paiements	/rappels-paiement , relancer , regler , DELETE	le retard est recalculé à chaque lecture ; tâche planifiée BullMQ upsertJobScheduler('relances- quotidiennes', { pattern: '0 9 * * *', tz: 'Africa/Dakar' }) (format cron : minute heure jour mois jour-de-semaine → 9 h 00 chaque jour) ; doitRelancer : 3 jours avant, le lendemain, puis toutes les semaines, 4 fois au plus
evenements	/evenements , piece-jointe , publier , annuler , reponses	réutilise la table annonces (type EVENEMENT) et AnnoncesService ; rappel automatique la veille à 18 h ; réponses oui/non des parents (la plus récente des tuteurs est retenue) ; pièce jointe PDF ou image reconnue à ses octets
utilisateurs	/utilisateurs , reinitialiser-mot- de-passe , moi/mot-de-passe	la direction crée les comptes ; motDePasseProvisoire() sans caractères ambigus (0/O, 1/I) ; 10 caractères minimum
plateforme	/plateforme/tableau-de-bord , ecoles , paiements , suspendre , reactiver , direction/:id/reinitialiser ; /abonnement (direction)	réservé à SUPER_ADMIN ; créer une école crée aussi son année scolaire, ses 3 trimestres et le compte de direction ; les élèves sont seulement comptés , jamais lus

Module	Routes principales	Ce qu'il a de particulier
audit	GET /audit	consultation du journal par la direction

C.9 Les tests (`api/test` et `*.spec.ts`)

Tests unitaires : voir C.5. Configuration `vitest.config.ts` : `include: ['**/*.spec.ts'], globals: true`.

Tests de bout en bout (`vitest.config.e2e.ts`) : variables de test (base locale, faux fournisseurs `simulation`, délais courts), `setupFiles: ['./test/preparation.ts']` (un préfixe Redis différent par fichier pour que les tests ne se gênent pas). Structure d'un fichier `e2e` :

```
describe('Annonces et absences (e2e)', () => {
  let app: INestApplication<App>;
  beforeAll(async () => {
    const moduleRef = await Test.createTestingModule({ imports: [AppModule] }).compile();
    app = moduleRef.createNestApplication();
    configurerApplication(app); // mêmes réglages qu'en vrai (C.1)
    await app.init();
    prisma = app.get(PrismaService); // récupère un service pour préparer les
données
    await prisma.ecole.create({ data: { id: ecoleId, nom: 'École des annonces' } }); // école de test
isolée
    ... // jetons signés directement avec JwtService
  });
  it('...', async () => {
    const reponse = await request(app.getHttpServer()) // supertest : vraie requête HTTP
      .post('/api/annonces')
      .set('Authorization', `Bearer ${jetons.secretariat}`)
      .send({ type: 'LIBERATION_ANTICIPEE', ... })
      .expect(201); // code attendu
    ...
  });
  afterAll(async () => { await app.close(); });
});
```

La fonction `attendre(lire, ok)` relit la base toutes les 100 ms jusqu'à ce que la file BullMQ ait fini (les envois sont asynchrones).

Le test de sécurité (`securite.e2e-spec.ts`) utilise `DiscoveryService` de NestJS pour **lister toutes les routes** de l'application, avec leurs étiquettes `@Public` / `@Roles` / `@ParamEleve`, et vérifie :

- toute route sans `@Roles` est dans la liste justifiée `OUVERTES_A_TOUS_CONNECTES` ;
- un parent ne peut pas lire l'enfant d'un autre ;
- les limites de tentatives, les en-têtes, le consentement, l'export et l'effacement.

C.10 L'image Docker de l'API (`api/Dockerfile`)

```
FROM node:22-alpine AS base # image de départ : Node 22 sur Alpine Linux ; « base » = nom de
l'étape
RUN corepack enable # active pnpm (livré avec Node via corepack)
WORKDIR /app # dossier de travail dans l'image
ENV CI=1

FROM base AS construction # étape 2, à partir de « base »
```

```

COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
COPY api/package.json api/          # d'abord les manifestes : Docker garde en cache l'installation
COPY web/package.json web/          # tant qu'ils ne changent pas (constructions bien plus rapides)
COPY mobile/package.json mobile/
COPY api/prisma api/prisma          # nécessaire au postinstall (prisma generate)
COPY api/prisma7.config.ts api/
RUN pnpm install --frozen-lockfile --filter @suivi-eleve/api... # versions exactes du lockfile ; api et
ses dépendances
COPY api/ api/                      # puis le code
RUN pnpm --filter @suivi-eleve/api exec prisma generate --config prisma7.config.ts \
  && pnpm --filter @suivi-eleve/api build          # \ : la commande continue à la ligne suivante

FROM base AS execution              # étape finale
ENV NODE_ENV=production API_PORT=3100 STOCKAGE_DIR=/donnees/stockage
COPY --from=construction --chown=node:node /app /app # reprend le résultat de l'étape « construction »
RUN mkdir -p /donnees/stockage && chown -R node:node /donnees
WORKDIR /app/api
USER node                          # s'exécute sans les droits administrateur (sécurité)
EXPOSE 3100                        # port documenté
CMD ["sh", "-c", "pnpm exec prisma migrate deploy --config prisma7.config.ts && exec node dist/main.js"]
# au démarrage : applique les migrations en attente, PUIS lance l'API (exec : remplace le shell)

```

Partie D — Le site web (Next.js)

D.1 La configuration

web/package.json

Scripts : `dev` = `next dev -p 3001` (serveur de développement sur le port 3001), `build` = `next build`, `start`, `lint` = `eslint`, `typecheck` = `next typegen && tsc --noEmit` (`next typegen` génère les types des pages, comme `PageProps<'annonces'>`). Dépendances : `next`, `react`, `react-dom` ; outils : `tailwindcss`, `@tailwindcss/postcss`, `eslint-config-next`, `typescript`.

web/tsconfig.json

Particularités : `"paths": { "@/*": ["./*"] }` (l'alias `@/` de B.1), `"jsx": "react-jsx"`, `"plugins": [{ "name": "next" }]`.

web/next.config.ts

```

const envRacine = resolve(process.cwd(), '../.env'); // le .env commun à la racine
if (existsSync(envRacine)) process.loadEnvFile(envRacine); // chargé s'il existe

const PROD = process.env.NODE_ENV === 'production';

const CSP = [ // Content-Security-Policy : d'où la page a le droit de
charger
  "default-src 'self'", // par défaut : seulement le site lui-même
  `script-src 'self' 'unsafe-inline'${PROD ? '' : "'unsafe-eval'"}`,
  "img-src 'self' data: blob:", // images du site, ou générées dans la page
  "frame-ancestors 'none'", // interdit d'afficher le site dans un cadre (anti-
hameçonnage)
  "form-action 'self'", "object-src 'none'", ...
].join('; ');

```

```

const nextConfig: NextConfig = {
  output: 'standalone', // serveur autonome pour Docker (voir F.2)
  outputFileTracingRoot: resolve(process.cwd(), '..'), // monorepo : chercher les fichiers depuis la
  racine
  poweredByHeader: false, // ne pas annoncer « X-Powered-By: Next.js »
  experimental: { serverActions: { bodySizeLimit: '6mb' } }, // envoi de photos et pièces jointes
  headers() { // en-têtes HTTP ajoutés à chaque réponse
    return [
      { source: '/:path*', headers: [ { key: 'Content-Security-Policy', value: CSP },
        { key: 'X-Frame-Options', value: 'DENY' }, { key: 'X-Content-Type-Options', value: 'nosniff' },
        ..., ...(PROD ? [{ key: 'Strict-Transport-Security', value: 'max-age=31536000; includeSubDomains'
      ] : []) ] },
      { source: '/sw.js', headers: [ { key: 'Cache-Control', value: 'no-cache, no-store, must-revalidate'
    ] } ],
    // le service worker est toujours relu : les mises à jour arrivent
  ];
},
};
export default nextConfig;

```

postcss.config.mjs branche Tailwind (plugins: { '@tailwindcss/postcss': {} }). eslint.config.mjs reprend les règles de Next (next/core-web-vitals , next/typescript).

D.2 La session : cookies et proxy.ts

web/lib/session.ts

```

export const COOKIE_ACCES = 'suivi_acces'; // jeton d'accès (15 min)
export const COOKIE_RAfraichissement = 'suivi_rafraichissement'; // jeton de rafraîchissement (30 jours)

const SECURISES =
  process.env.NODE_ENV === 'production' &&
  process.env.COOKIES_SECURISES !== 'non';

export const optionsCookie = (maxAgeSecondes: number) => ({
  httpOnly: true, // le JavaScript de la page ne peut PAS le lire (anti-vol)
  secure: SECURISES, // envoyé seulement en HTTPS (en production)
  sameSite: 'lax' as const, // pas envoyé par un autre site (anti-CSRF)
  path: '/',
  maxAge: maxAgeSecondes,
});

export const API_URL = process.env.API_URL ?? 'http://localhost:3100/api';

export function expirationJeton(jeton: string): number | null {
  try {
    const charge = JSON.parse(
      Buffer.from(jeton.split('.')[1], 'base64url').toString('utf8'), // un JWT = entête.CHARGE.signature
    ) as { exp?: number };
    return charge.exp ? charge.exp * 1000 : null; // exp est en secondes → millisecondes
  } catch {
    return null;
  }
}

```

La charge d'un JWT est lisible par tous (seulement **signée**, pas chiffrée) : le site la lit pour savoir **quand** renouveler, mais seule l'API la **vérifie**.

```

const MARGE_MS = 60 * 1000;

export async function proxy(request: NextRequest) { // appelée avant chaque page
  const { pathname, search } = request.nextUrl;
  const surConnexion = pathname === '/connexion';
  const acces = request.cookies.get(COOKIE_ACCES)?.value;
  const rafraichissement = request.cookies.get(COOKIE_RAfraichissement)?.value;

  const expiration = acces ? expirationJeton(acces) : null;
  if (expiration && expiration - Date.now() > MARGE_MS) { // jeton valable encore plus d'une minute
    return surConnexion
      ? NextResponse.redirect(new URL(accueilDuRole(roleJeton(acces!) ?? ''), request.url)) // déjà
connecté
      : NextResponse.next(); // on continue vers la page
  }
  if (rafraichissement) {
    const session = await rafraichir(rafraichissement); // POST /api/auth/rafraichir
    if (session) {
      request.cookies.set(COOKIE_ACCES, session.jetonAcces); // utilisable dès cette requête côté
serveur
      ...
      const reponse = surConnexion ? NextResponse.redirect(...)
        : NextResponse.next({ request: { headers: request.headers } });
      reponse.cookies.set(COOKIE_ACCES, session.jetonAcces, optionsCookie(DUREE_ACCES_S)); // et pour le
navigateur
      reponse.cookies.set(COOKIE_RAfraichissement, ...);
      return reponse;
    }
  }
  if (surConnexion) return NextResponse.next();
  const connexion = new URL('/connexion', request.url);
  connexion.searchParams.set('suite', pathname + search); // revenir ici après la connexion
  const reponse = NextResponse.redirect(connexion);
  reponse.cookies.delete(COOKIE_ACCES); reponse.cookies.delete(COOKIE_RAfraichissement);
  return reponse;
}

export const config = {
  matcher: [ '/(?!(?!_next/static|_next/image|sw\\.js$|manifest\\.webmanifest$|hors-ligne\\.html$|\\..*\\.(?:png|svg|ico)$).*)' ],
  // (?!) = « sauf » : le proxy ne s'applique pas aux fichiers statiques, au service worker, au
manifeste...
};

```

Règle du projet : tout nouveau fichier public autre que `.png/.svg/.ico` doit être ajouté à cette exception.

D.3 Parler à l'API : `web/lib/api.ts`

```

import 'server-only'; // erreur de construction si ce fichier est importé dans un composant client

export class ErreurApi extends Error {
  constructor(
    readonly statut: number,
    message: string,
  ) {
    super(message);
  }
}

```

```

}

export async function transmettreIp(entetes: Headers): Promise<void> {
  const ip = (await headers()).get('x-forwarded-for'); // IP du visiteur, reçue de Caddy
  if (ip) entetes.set('X-Forwarded-For', ip); // transmise à l'API (limites, audit)
}

export async function appelApi(
  chemin: string,
  init: RequestInit = {},
): Promise<Response> {
  const jeton = (await cookies()).get(COOKIE_ACCES)?.value; // cookies() : lecture côté serveur
  // (asynchrone en Next 16)
  if (!jeton) redirect('/connexion'); // redirect() interrompt et change de page
  const entetes = new Headers(init.headers);
  entetes.set('Authorization', `Bearer ${jeton}`);
  await transmettreIp(entetes);
  if (init.body && !(init.body instanceof FormData))
    entetes.set('Content-Type', 'application/json');
  const reponse = await fetch(`${API_URL}${chemin}`, {
    ...init,
    headers: entetes,
    cache: 'no-store',
  });
  // cache: 'no-store' : toujours des données fraîches (jamais de cache pour des données d'élèves)
  if (reponse.status === 401) redirect('/connexion');
  if (!reponse.ok) {
    const corps: unknown = await reponse.json().catch(() => null);
    throw new ErreurApi(reponse.status, messageErreur(corps, reponse.status));
  }
  return reponse;
}

export async function lireApi<T>(chemin: string): Promise<T> {
  // pour les pages
  try {
    return (await (await appelApi(chemin)).json()) as T;
  } catch (e) {
    if (e instanceof ErreurApi && e.statut === 403) redirect('/accus-refuse');
    throw e;
  }
}

// lireApiOuNull : 404/400 → null (la page affiche alors notFound())
// envoyerApi(chemin, 'POST'|'PUT'|'PATCH'|'DELETE', corps) : pour les actions ; 204 → undefined

export interface EtatFormulaire {
  erreur?: string;
  succes?: string;
}

export async function tenter(
  action: () => Promise<unknown>,
): Promise<EtatFormulaire | null> {
  try {
    await action();
    return null;
  } catch (e) {
    if (e instanceof ErreurApi) return { erreur: e.message };
    throw e;
  } // erreur API → message du formulaire
}

```

Autres fichiers de `web/lib` :

- `profil.ts` : `profilCourant = cache(() => lireApi<Profil>('/auth/moi'))` — `cache` de React : lu **une seule fois** par requête même si plusieurs composants le demandent ; `peutGererDossiers()` .
- `types.ts` (≈ 1000 lignes) : les types des réponses de l'API (`Eleve` , `Annonce` ...), les libellés français (`LIBELLES_ROLE` , `LIBELLES_STATUT_ANNONCE` ...), des fonctions d'affichage (`dateFr` , `dateHeureFr`) et `accueilDuRole(role)` (page d'accueil de chaque rôle).
- `pays.ts` : copie de la table des pays + `telephoneE164` .
- `theme.ts` / `theme-serveur.ts` : nom du cookie `theme` et lecture du thème côté serveur.
- `telechargement.ts` : `relayerFichier(chemin)` renvoie au navigateur un fichier produit par l'API (Excel, PDF) en gardant `content-type` et `content-disposition` (nom du fichier).

D.4 Les mises en page (`layout.tsx`)

`web/app/layout.tsx` (racine)

```
const geistSans = Geist({ variable: '--font-geist-sans', subsets: ['latin'] }); // police optimisée par
Next

export const metadata: Metadata = {
  // balises <title>, <meta>... de toutes les pages
  title: 'Suivi_eleve',
  description: 'Suivi des élèves pour les parents',
  applicationName: 'Suivi_eleve',
  appleWebApp: {
    capable: true,
    title: 'Suivi_eleve',
    statusBarStyle: 'default',
  }, // plein écran sur iPhone
};

export async function generateViewport(): Promise<Viewport> {
  // couleur de la barre du navigateur
  const theme = await themeCourant();
  return {
    themeColor: COULEUR_BARRE[theme],
    colorScheme: theme === 'sombre' ? 'dark' : 'light',
  };
}

export default async function RootLayout({ children }: LayoutProps<'>) {
  // children = la page affichée
  return (
    <html
      lang="fr"
      data-theme={await themeCourant()} // data-theme : active le mode sombre du CSS
      className={` ${geistSans.variable} ${geistMono.variable} h-full antialiased` }
    >
      <body className="flex min-h-full flex-col">
        <EnregistrementServiceWorker /> // enregistre sw.js (D.9)
        <InvitationInstallation /> // bandeau « Installer l'application »
        {children}
      </body>
    </html>
  );
}
```

En JSX on écrit `className` (et non `class`), et les attributs dynamiques entre accolades.

web/app/page.tsx

```
export default async function Accueil() {
  redirect(accueilDuRole((await profilCourant()).role)); // « / » → /eleves, /parent ou /plateforme
}
```

web/app/(admin)/layout.tsx

```
export default async function LayoutAdmin({ children }: LayoutProps<'>') {
  const profil = await profilCourant();
  if (profil.role === 'PARENT') redirect('/parent'); // chacun dans son espace
  if (profil.role === 'SUPER_ADMIN') redirect('/plateforme');
  return (
    <div className="flex min-h-full flex-1 flex-col">
      <Bandeau
        accueil="/eleves"
        espace={LIBELLES_ROLE[profil.role]}
        utilisateur={` ${profil.prenoms} ${profil.nom}`}
        compte="/compte"
      />
      <Navigation role={profil.role} /> // menu selon le rôle
      {profil.role === 'ADMIN' && <BandeauAbonnement />} // affiché seulement
      pour la direction (B.7 &&)
      <main className="mx-auto w-full max-w-6xl flex-1 px-4 py-6 sm:py-8">
        {children}
      </main>
    </div>
  );
}
```

(parent)/layout.tsx et (plateforme)/layout.tsx font la même chose pour leur espace ;
(parent)/parent/(espace)/layout.tsx ajoute la vérification du **consentement** (sinon
/parent/consentement).

D.5 Une page complète : les annonces (web/app/(admin)/annonces)

page.tsx — la liste (composant serveur)

```
export const metadata: Metadata = { title: 'Annonces · Suivi_eleve' }; // titre de l'onglet

const COULEUR: Record<StatutAnnonce, 'gris' | 'vert' | 'orange'> = {
  BROUILLON: 'gris',
  PROGRAMMEE: 'orange',
  ENVOYEE: 'vert',
  ANNULEE: 'gris',
};

export default async function PageAnnonces(props: PageProps<'>') {
  // composant ASYNC : serveur
  const page = parametre((await props.searchParams).page) ?? '1'; // ?page=2 (searchParams asynchrone)
  const [annonces, gestion] = await Promise.all([
    // deux lectures en parallèle (B.9)
    lireApi<Page<AnnonceListe>>(` /annonces?page=${page}`),
    peutGererDossiers(),
  ]);
  return (
```

```

<>
{' '}
{/* fragment : regroupe sans ajouter de balise */}
<EnTete
  titre="Annonces"
  sousTitre="Libérations anticipées et absences de cours"
  actions={
    gestion && (
      <>
        <Link
          className={styles.boutonSecondaire}
          href="/annonces/nouvelle?type=PAS_DE_COURS"
        >
          Pas de cours
        </Link>
        <Link
          className={styles.bouton}
          href="/annonces/nouvelle?type=LIBERATION_ANTICIPEE"
        >
          Libération anticipée
        </Link>
      </>
    )
  }
/>
<Tableau
  entetes={['Annonce', 'Motif', 'Statut', 'Familles', 'Lu', 'Par']}
  vide={
    annonces.total === 0 ? 'Aucune annonce pour le moment.' : undefined
  }
>
  {annonces.elements.map((a) => (
    // une ligne par annonce
    <tr key={a.id} className="hover:bg-marque-50/60">
      {' '}
      {/* key : identifiant unique exigé par React dans une liste */}
      <td className={cellule}>
        <Link className={styles.lien} href={`/${annonces}/${a.id}`}>
          {a.titre}
        </Link>
      </td>
      ...
      <td className={cellule}>
        {a.familles
          ? `${Math.round((a.lues / a.familles) * 100)} %`
          : '-'}
        </td>
      </tr>
    )))
</Tableau>
<Pagination
  page={annonces.page}
  pages={annonces.pages}
  total={annonces.total}
  chemin="/annonces"
  parametres={{}
/>
</>
);
}

```

`<Link>` (de `next/link`) : lien interne qui change de page **sans recharger** tout le site.

nouvelle/page.tsx — le formulaire (serveur) qui contient des champs (client)

```
export default async function PageNouvelleAnnonce(props: PageProps<'/annonces/nouvelle'>) {
  const type = parametre((await props.searchParams).type) === 'PAS_DE_COURS' ? 'PAS_DE_COURS' :
  'LIBERATION_ANTICIPEE';
  const classes = await lireApi<Classe[]>('/classes'); // la liste des classes, lue côté
  serveur
  return (<>
    <EnTete titre="Prévenir les familles" ... />
    <Carte>
      <FormulaireAction action={creerAnnonce} libelle="Prévenir les familles" libelleEnCours="Envoi..."
        confirmation="Confirmer l'envoi aux familles concernées ?" className="flex flex-
col gap-6">
        <ChampsAnnonce typeInitial={type} classes={classes} /> /* composant client, reçoit les
données en props */
      </FormulaireAction>
    </Carte>
  </>);
}
```

nouvelle/champs-annonce.tsx — composant client (interactif)

```
'use client'; // ce composant s'exécute dans le navigateur
export function ChampsAnnonce({ typeInitial, classes }: { typeInitial: TypeAnnonce; classes: Classe[] })
{
  const [type, setType] = useState<TypeAnnonce>(typeInitial); // état : le type choisi
  const [cible, setCible] = useState<'ECOLE' | 'CLASSES'>('CLASSES');
  const [motif, setMotif] = useState('');
  const [envoi, setEnvoi] = useState<'immédiat' | 'programme'>('immédiat');
  const liberation = type === 'LIBERATION_ANTICIPEE';
  return (
    <div className="flex flex-col gap-5">
      <fieldset> ...
        <input type="radio" name="type" value={valeur} checked={type === valeur}
          onChange={() => setType(valeur)} /> /* changer l'état redessine les
champs */
      </fieldset>
      {liberation ? <Saisie type="time" name="heure" ... /> : <Liste name="creneau">...</Liste>}
      ...
    </div>
  );
}
```

`useState(valeurInitiale)` renvoie `[valeur, fonctionPourLaChanger]`. Chaque champ a un attribut `name` : c'est sous ce nom qu'il sera lu dans le `FormData` de la Server Action.

actions.ts — les Server Actions

```
'use server'; // toutes les fonctions exportées s'exécutent sur le
serveur
type Etat = EtatFormulaire | null;

const texte = (d: FormData, cle: string) => { // lit un champ texte, vide → undefined
  const v = d.get(cle);
  return typeof v === 'string' && v.trim() !== '' ? v.trim() : undefined;
};

export async function creerAnnonce(_e: Etat, d: FormData): Promise<Etat> { // (état précédent, données
```

```

du formulaire)
const type = texte(d, 'type');
const cible = texte(d, 'cible');
const programme = d.get('envoi') === 'programme';
const quand = texte(d, 'programmeLe');
if (programme && !quand) return { erreur: "Choisissez la date et l'heure d'envoi." };

let id = '';
const erreur = await tenter(async () => {
  const annonce = await envoyerApi<Annonce>('/annonces', 'POST', {
    type, cible,
    classeIds: cible === 'CLASSES' ? d.getAll('classeIds') : undefined, // getAll : cases cochées
    date: texte(d, 'date'), ...,
    programmeLe: programme ? `${quand}:00Z` : undefined, // heure de Dakar = UTC
  });
  id = annonce.id;
});
if (erreur) return erreur; // affiché par le formulaire
redirect(`/annonces/${id}`); // succès : page de suivi de l'annonce
}

export async function annulerAnnonce(id: string): Promise<Etat> {
  const erreur = await tenter(() => envoyerApi(`/annonces/${id}/annuler`, 'POST'));
  if (erreur) return erreur;
  refresh(); // Next 16 : relit la page affichée avec les
nouvelles données
  return { succes: 'Envoi annulé.' };
}

```

web/components/formulaire-action.tsx — le formulaire réutilisable

```

'use client';
export function FormulaireAction({ action, libelle, libelleEnCours = 'Enregistrement...', style = 'bouton',
  confirmation, reinitialiserSiSucces = false, className, children }: {
  ... }) {
  const [etat, envoyer, enCours] = useActionState(action, null);
  // useActionState : [dernier résultat de l'action, fonction pour la lancer, vrai pendant l'envoi]
  const formulaire = useRef<HTMLFormElement>(null); // référence vers la balise <form>

  useEffect(() => { // après chaque nouvel état...
    if (reinitialiserSiSucces && etat?.succes) formulaire.current?.reset(); // ... vider les champs si
demandé
  }, [etat, reinitialiserSiSucces]); // tableau des dépendances : quand relancer
l'effet

  return (
    <form ref={formulaire} className={className ?? 'flex flex-col gap-4'}
      onSubmit={(e) => {
        e.preventDefault(); // empêche le rechargement de la page
        if (confirmation && !window.confirm(confirmation)) return; // boîte « OK / Annuler »
        const donnees = new FormData(e.currentTarget); // tous les champs du formulaire
        startTransition(() => envoyer(donnees)); // lance la Server Action
      }}>
      {children}
      {etat?.erreur && <Alerte>{etat.erreur}</Alerte>}
      {etat?.succes && <Alerte type="succes">{etat.succes}</Alerte>}
      <div><button type="submit" disabled={enCours} className={styles[style]}>
        {enCours ? libelleEnCours : libelle}</button></div>
    </form>
  )
}

```

```
);
}
```

Tous les formulaires du site utilisent ce composant : la saisie n'est **pas perdue** en cas d'erreur, et le bouton est désactivé pendant l'envoi (pas de double envoi).

D.6 La connexion ([web/app/connexion](#))

`page.tsx` affiche deux onglets (**Parents** / **Personnel de l'école**, selon `?espace=`). `actions.ts` :

- `cheminSur(suite)` n'accepte qu'un chemin interne commençant par `/` (empêche une redirection vers un site pirate).
- `seConnecter` : `POST /auth/connexion` puis `ouvrirSession` (pose les deux cookies) et redirection.
- `demanderCode` : convertit le numéro en E.164 avec le pays choisi, retient ce pays dans un cookie, `POST /auth/otp/demande` , puis affiche l'étape « code ».
- `verifierCode` : `POST /auth/otp/verification` → cookies → `/parent` .
- `seDeconnecter` : prévient l'API (révocation du jeton) puis supprime les cookies.

D.7 Les composants ([web/components](#))

Fichier	Rôle
<code>ui.tsx</code>	Briques communes : <code>styles</code> (classes Tailwind des champs et boutons), <code>Champ</code> , <code>Saisie</code> , <code>Liste</code> , <code>Zone</code> , <code>Alerte</code> , <code>Carte</code> , <code>EnTete</code> , <code>Tableau</code> , <code>Badge</code> , <code>Pagination</code> , <code>parametre()</code>
<code>formulaire-action.tsx</code>	Formulaire relié à une Server Action (D.5)
<code>bandeau.tsx</code>	Bandeau du haut en dégradé : nom de l'espace, utilisateur, bouton 🌞 / 🌙 , déconnexion
<code>navigation.tsx</code>	Menu du personnel filtré par rôle ; pastilles sur grand écran, bouton « Menu » dépliant sur téléphone (<code>usePathname()</code> donne la page active)
<code>choix-theme.tsx</code>	Bouton 🌞 / 🌙 : change <code>data-theme</code> sur <code><html></code> , la couleur de la barre et le cookie <code>theme</code> (1 an)
<code>abonnement.tsx</code>	Bandeau d'avertissement de fin d'abonnement pour la direction
<code>parent.tsx</code>	Menu et briques de l'espace parents
<code>plateforme.tsx</code>	Briques de l'espace concepteur (liste des écoles, états)
<code>champs-eleve.tsx</code>	Champs du formulaire élève (inscription et modification)
<code>appareils.tsx</code>	Affichage et formulaires des appareils
<code>installation.tsx</code>	Enregistre le service worker et propose l'installation (<code>beforeinstallprompt</code> sur Chrome/Android, explication « Partager → Sur l'écran d'accueil » sur iPhone) ; <code>localStorage</code> retient si le bandeau a été masqué
<code>notifications-push.tsx</code>	Active / coupe les notifications du site sur l'appareil (D.9)
<code>rafraichissement-auto.tsx</code>	Relit la page régulièrement (suivi en temps réel d'un envoi)

Extrait de `choix-theme.tsx` :

```
const basculer = () => {
  document.documentElement.dataset.theme = suivant; // <html data-theme="sombre">
  for (const meta of document.querySelectorAll<HTMLMetaElement>(
    'meta[name="theme-color"]',
  ))
    meta.content = COULEUR_BARRE[suivant]; // couleur de la barre du navigateur
  const securise = location.protocol === 'https:' ? ';' : 'secure' : '';
  document.cookie = `${COOKIE_THEME}=${suivant}; path=/; max-age=31536000; samesite=lax${securise}`;
  setTheme(suivant);
};
```

`document` (le DOM) n'existe que dans le navigateur : voilà pourquoi ce composant est `'use client'`.

D.8 Le style : `web/app/globals.css`

```
@import 'tailwindcss';                                /* charge Tailwind */

@theme inline { --font-sans: var(--font-geist-sans); } /* police par défaut */

@theme {                                                /* nouvelles couleurs → classes bg-marque-600, text-soleil-400... */
  --color-marque-50: oklch(97% 0.014 254.604); /* oklch(luminosité chroma teinte) : format moderne de couleur */
  ...
  --color-carte: #fff;                                /* → classe bg-carte */
}

:root { color-scheme: light; --fond-page: #f4f7fc; --degrade-bleu: #193cb8; ... } /* variables CSS */

:root[data-theme='sombre'] {                            /* quand <html data-theme="sombre"> */
  color-scheme: dark;
  --color-carte: #0d1322;
  --color-slate-900: #f1f5f9;                          /* le texte « foncé » devient clair */
  --color-marque-50: #0c1730;                          /* le fond « pâle » devient bleu nuit */
  ...
}
```

Le reste du fichier définit les classes `fond-bandeau`, `fond-degrade`, `contour-degrade` (dégradés bleus) et le fond de page. **Inverser les nuances** évite d'écrire deux classes (`dark:`) partout.

D.9 Le site installable : manifest.ts , sw.js , notifications-push.tsx

`app/manifest.ts` renvoie un objet (nom, `display: 'standalone'` = sans barre d'adresse, couleurs, icônes 192/512 et « masquable ») que Next publie à `/manifest.webmanifest`.

public/sw.js (JavaScript pur, exécuté par le navigateur) :

```
const CACHE = 'suivi-eleve-v2'; // changer le nom = vider l'ancien cache
self.addEventListener('install', (evenement) => { // self = le service worker
  evenement.waitUntil(caches.open(CACHE)
    .then((cache) => cache.addAll(['/hors-ligne.html', '/icones/icone-192.png'])) // seulement ces
fichiers
    .then(() => self.skipWaiting())); // s'active sans attendre
});
```

```

self.addEventListener('activate', ...); // supprime les anciens caches, prend le
contrôle
self.addEventListener('fetch', (evenement) => {
  if (evenement.request.mode !== 'navigate') return; // seulement les changements de page
  evenement.respondWith(fetch(evenement.request)
    .catch(() => caches.match('/hors-ligne.html'))); // réseau absent → page hors ligne
});
self.addEventListener('push', (evenement) => { // notification reçue
  const message = evenement.data ? evenement.data.json() : {};
  evenement.waitUntil(self.registration.showNotification(message.titre || 'Suivi_eleve', {
    body: message.texte, icon: '/icones/icone-192.png', tag: donnees.lotId, // tag : pas de doublon
    data: { url: donnees.url || '/parent/messages' } }));
});
self.addEventListener('notificationclick', ...); // ouvre le message (dans une fenêtre existante
si possible)

```

`components/notifications-push.tsx` : vérifie la compatibilité (`'serviceWorker' in navigator` && `'PushManager' in window`), détecte l'iPhone non installé, demande la permission, s'abonne avec la clé VAPID (`cleVersOctets` convertit la clé base64url en octets), puis envoie l'abonnement au serveur par une Server Action (`abonnerNotifications`).

D.10 Les routes de téléchargement

```

// web/app/telechargements/eleves/route.ts
export function GET(request: NextRequest) {
  // un fichier route.ts exporte une fonction par méthode HTTP
  return relayFichier(
    `/eleves/export?${request.nextUrl.searchParams.toString()}`,
  );
}

```

Les autres (`bulletin/[eleveId]/[periodeId]` , `donnees-eleve/[id]` , `etiquettes` , `modele-import` , `photo-appareil/[id]` , `piece-jointe-evenement/[id]`) sont identiques : le navigateur ne connaît jamais le jeton ni l'adresse de l'API.

D.11 Toutes les pages

Espace du personnel — `app/(admin)/` (chaque dossier a `page.tsx` et souvent `actions.ts`) :

Adresse	Contenu
<code>/eleves</code> , <code>/eleves/[id]</code> , <code>/eleves/[id]/modifier</code> , <code>/eleves/nouveau</code> , <code>/eleves/import</code>	liste filtrable + export Excel/CSV, fiche complète (tuteurs, classe, appareils, absences...), inscription, import
<code>/tuteurs</code> , <code>/tuteurs/[id]</code>	parents et responsables
<code>/classes</code> , <code>/classes/[id]</code>	classes, enseignant principal, matières enseignées
<code>/absences</code>	appel et justification
<code>/comportements</code> , <code>/comportements/nouveau</code>	signalements, validation des cas graves, convocations
<code>/resultats</code> , <code>/resultats/saisie</code> , <code>/resultats/synthese</code> , <code>/resultats/decisions</code> , <code>/resultats/matieres</code>	saisie des moyennes, synthèse, publication, décisions de fin d'année, matières
<code>/paiements</code>	rappels de paiement, relances, « Régulé »

Adresse	Contenu
/annonces , /annonces/nouvelle , /annonces/[id]	pas de cours / libération anticipée et suivi de lecture
/evenements , /evenements/nouveau , /evenements/[id]	événements, pièce jointe, réponses des familles
/appareils , /appareils/nouveau , /appareils/[id] , /appareils/scan/[code]	appareils, photos, signalements, page ouverte en scannant une étiquette
/notifications , /notifications/modeles	journal des envois, statistiques, modèles de messages
/personnel	comptes de l'équipe (direction)
/journal	journal d'audit (direction)
/abonnement	abonnement de l'école (direction)
/compte	changer son mot de passe
/acces-refuse	page affichée après un refus 403

Espace parents — `app/(parent)/parent/` : `consentement`, et dans `(espace)` : `accueil`, `messages` et `messages/[id]`, `resultats`, `absences`, `comportement`, `paiements`, `appareils`, `evenements` et `evenements/[id]` (répondre oui/non), `preferences` (canaux et notifications de l'appareil). `actions.ts` contient les actions du parent (justifier une absence, répondre, déclarer une perte, préférences, abonnement push).

Espace concepteur — `app/(plateforme)/plateforme/` : tableau de bord, `ecoles`, `ecoles/nouvelle`, `ecoles/[id]` (paiements, suspension, compte de direction), `compte`.

Partie E — L'application mobile (Expo)

E.1 La configuration

- `package.json` : `"main": "expo-router/entry"` (le point d'entrée est le routeur d'Expo) ; scripts `start` (`expo start`), `android`, `ios`, `web`, `typecheck`, `lint` (`expo lint`), `test` (`vitest run`).
- `app.json` : nom, icône, identifiants Android/iOS, plugins Expo (caméra, notifications...).
- `app.config.ts` : **complète** `app.json` au moment de la construction. Les fichiers Firebase (`google-services.json`) ne sont jamais dans Git : leur chemin vient de variables d'environnement.

```
export default ({ config }: ConfigContext): ExpoConfig => {
  const googleServices = process.env.GOOGLE_SERVICES_JSON;
  return { ...config,
    android: { ...config.android, ...(googleServices ? { googleServicesFile: googleServices } :
  {} ) },
    // ajout conditionnel : ...(condition ? { clé: valeur } : {})
    ios: { ..., infoPlist: { ..., ITSAAppUsesNonExemptEncryption: false } } }; // déclaration Apple
    sur le chiffrement
  };
};
```

- `eas.json` : profils de construction EAS. `base` fixe `EXPO_PUBLIC_API_URL` (adresse de l'API ; les variables `EXPO_PUBLIC_*` sont incluses dans l'application) ; `development` (client de développement, APK), `preview` (APK à installer directement), `production` (numéro de version incrémenté automatiquement, envoi au Play Store en test interne).

E.2 La racine : `src/app/_layout.tsx`

```
function Navigation() {
  const { chargement, utilisateur } = useSession(); // contexte de session (E.4)
  const { couleurs } = useTheme(); // contexte de thème (E.5)
  if (chargement) {
    // reprise de la session enregistrée en cours
    return (
      <View
        style={{
          flex: 1,
          justifyContent: 'center',
          backgroundColor: couleurs.fond,
        }}
      >
        <ActivityIndicator size="large" color={couleurs.primaire} />
      </View>
    );
  }
  const parent = utilisateur?.role === 'PARENT';
  return (
    <>
      <StatusBar style="light" /> {/* heure et batterie en blanc */}
      <Stack
        screenOptions={{
          headerShown: false,
          contentStyle: { backgroundColor: couleurs.fond },
        }}
      >
        <Stack.Protected guard={!utilisateur}>
          { ' ' }
          {/* visibles seulement sans session */}
          <Stack.Screen name="connexion" />
          <Stack.Screen name="connexion-personnel" />
        </Stack.Protected>
        <Stack.Protected guard={parent}>
          <Stack.Screen name="(parent)" />
        </Stack.Protected>
        <Stack.Protected guard={Boolean(utilisateur) && !parent}>
          <Stack.Screen name="(personnel)" />
        </Stack.Protected>
      </Stack>
    </>
  );
}

export default function Racine() {
  return (
    <SafeAreaProvider>
      { ' ' }
      {/* zones sûres : encoche, barre du bas */}
      <FournisseurTheme>
        <FournisseurSession>
          <Navigation />
        </FournisseurSession>
      </FournisseurTheme>
    </SafeAreaProvider>
  );
}
```

Le style React Native s'écrit en objet JavaScript ({ flex: 1, justifyContent: 'center' }): flex: 1 = prendre toute la place, justifyContent = alignement vertical (dans une colonne).

Fournisseurs (Providers) et contexte : un FournisseurX place une valeur dans un **contexte React** ; tout composant à l'intérieur la lit avec useX(), sans la passer de parent en enfant.

E.3 Les espaces et les onglets

(parent)/_layout.tsx :

```
export default function EspaceParent() {
  useOuvertureDepuisPush(); // toucher une notification ouvre les messages (E.6)
  const u = useUtilisateur();
  const entete = useOptionsEntete(); // entête en dégradé, commune
  const { couleurs } = useTheme();
  if (u.consentement && !u.consentement.accepte) return <Consentement />; // rien avant l'accord du
  parent
  return (
    <FournisseurEnfants>
      {' '}
      { /* enfants du parent + enfant sélectionné */ }
      <Stack
        screenOptions={{
          ...entete,
          contentStyle: { backgroundColor: couleurs.fond },
          headerBackButtonDisplayMode: 'minimal',
        }}
      >
        <Stack.Screen name="(onglets)" options={{ headerShown: false }} />
        <Stack.Screen name="notification/[id]" options={{ title: 'Message' }} />
        ... resultats, comportement, paiements, absences, appareils, preferences
      </Stack>
    </FournisseurEnfants>
  );
}
```

(parent)/(onglets)/_layout.tsx crée la **barre d'onglets** du bas : Accueil 🏠, Messages 💬, Événements 📅, Réglages ⚙️.

```
const icone = (symbole: string) =>
  // fonction qui fabrique un composant
  function Icone({ focused }: { focused: boolean }) {
    // focused : onglet actif ?
    return (
      <Text style={{ fontSize: 22, opacity: focused ? 1 : 0.55 }}>
        {symbole}
      </Text>
    );
  };
<Tabs.Screen
  name="index"
  options={{ title: 'Accueil', tabBarIcon: icone('🏠') }}
/>;
```

E.4 La session et les appels à l'API

src/lib/api.ts

```
export const API_URL = (process.env.EXPO_PUBLIC_API_URL ?? 'http://localhost:3100/api').replace(/\/$/, '');
// .replace(/\/$/, '') : enlève un « / » final éventuel

let jetonAcces: string | null = null; // en mémoire seulement (jamais sur le disque)
let rafraichissementEnCours: Promise<string | null> | null = null;

export async function ouvrirSession(session: Session) {
  jetonAcces = session.jetonAcces;
  await ecrireSecret(CLE_RAfraICHISSEMENT, session.jetonRafraichissement); // trousseau chiffré du téléphone
}

function renouveler(): Promise<string | null> { // un seul renouvellement à la fois
  rafraichissementEnCours ??= reprendreSession() // si déjà en cours, on réutilise la même promesse
  .then((s) => s?.jetonAcces ?? null).catch(() => null)
  .finally(() => { rafraichissementEnCours = null; });
  return rafraichissementEnCours;
}

export async function appelBrut(chemin: string, init: RequestInit = {}, renouvele = false): Promise<Response> {
  const entetes = new Headers(init.headers);
  if (jetonAcces) entetes.set('Authorization', `Bearer ${jetonAcces}`);
  if (init.body) entetes.set('Content-Type', 'application/json');
  let reponse: Response;
  try { reponse = await fetch(`${API_URL}${chemin}`, { ...init, headers: entetes }); }
  catch { throw new ErreurApi(0, 'Pas de connexion. Vérifiez votre réseau.'); }
  if (reponse.status === 401 && !renouvele) { // jeton expiré :
    const nouveau = await renouveler(); // on le renouvelle...
    if (nouveau) return appelBrut(chemin, init, true); // ... et on refait l'appel UNE fois
    surSessionExpiree(); // sinon : retour à la connexion
  }
  if (!reponse.ok) { ... throw new ErreurApi(reponse.status, messageErreur(corps, reponse.status)); }
  return reponse;
}

export async function lire<T>(chemin: string): Promise<T> { return (await (await appelBrut(chemin)).json()) as T; }
export async function envoyer<T>(chemin: string, methode: 'POST' | 'PUT' | 'DELETE', corps?: unknown): Promise<T> { ... }
```

Le mobile appelle **directement** l'API (contrairement au site) et gère lui-même le renouvellement.

src/lib/session.tsx

FournisseurSession garde utilisateur et chargement dans l'état et expose : demanderCode, verifierCode, connexionPersonnel, deconnexion, consentir.

- Au lancement (useEffect), reprendreSession() relit le jeton du trousseau et rouvre la session.
- ouvrir(session) lit le profil (/auth/moi) puis inscrit le push **sans attendre** (void inscrirePush()).
- connexionPersonnel refuse le concepteur (il utilise seulement le site web).
- deconnexion désinscrit le push, ferme la session et vide le cache local.

- `useMemo(() => ({...}), [dépendances])` : ne recrée l'objet que si les dépendances changent (évite de redessiner inutilement tous les écrans).
- `utiliserUtilisateur()` garde le dernier utilisateur connu le temps que les écrans se ferment après une déconnexion.

`src/lib/requete.ts` — le hook de lecture

```
export function useRequete<T>(chemin: string | null): Requete<T> {
  // hook personnalisé (commence par « use »)
  const [donnees, setDonnees] = useState<T | null>(null);
  const [erreur, setErreur] = useState<string | null>(null);
  const [chargement, setChargement] = useState(chemin !== null);
  const derniere = useRef(0); // numéro de la dernière demande (ne redessine pas)

  const recharger = useCallback(async () => {
    if (!chemin) return;
    const numero = ++derniere.current;
    setChargement(true);
    try {
      const resultat = await lire<T>(chemin);
      if (numero !== derniere.current) return; // réponse périmée (ex. changement d'enfant) : ignorée
      setDonnees(resultat);
      setErreur(null);
    } catch (e) {
      if (numero !== derniere.current) return;
      setErreur(e instanceof ErreurApi ? e.message : 'Erreur inattendue.');
```

```

export type Couleurs = typeof CLAIR; // le type est déduit de l'objet
const SOMBRE: Couleurs = { primaire: '#60a5fa', fond: '#05070d', carte: '#0d1322', ... }; // mêmes clés obligatoires

export function degrade(teintes: string[], angle = 135): ViewStyle {
  const css = `linear-gradient(${angle}deg, ${teintes[0]}, ${teintes[1]} 55%, ${teintes[2]})`;
  return Platform.OS === 'web' // Platform.OS : 'android', 'ios' ou 'web'
    ? ({ backgroundColor: teintes[1], backgroundImage: css } as ViewStyle)
    : { backgroundColor: teintes[1], experimental_backgroundImage: css };
}

export function FournisseurTheme({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<Theme>('clair');
  useEffect(() => { // au démarrage : relire le choix enregistré
    void AsyncStorage.getItem(CLE_THEME).then((t) => (t === 'sombre' || t === 'clair') &&
    setTheme(t)).catch(() => undefined);
  }, []); // [] : une seule fois
  const valeur = useMemo(() => ({ theme, couleurs: PALETTES[theme],
    basculer: () => { const suivant = theme === 'sombre' ? 'clair' : 'sombre'; setTheme(suivant);
    void AsyncStorage.setItem(CLE_THEME, suivant).catch(() => undefined); } })),
  [theme]);
  return <Contexte.Provider value={valeur}>{children}</Contexte.Provider>;
}

export function useStyles<T>(fabrique: (couleurs: Couleurs) => T): T { // styles recalculés au
  changement de thème
  const { couleurs } = useTheme();
  return useMemo(() => fabrique(couleurs), [fabrique, couleurs]);
}

```

Chaque écran déclare en bas `const creerStyles = (couleurs: Couleurs) => StyleSheet.create({ ... })` et fait `const styles = useStyles(creerStyles)` : **aucune couleur en dur**.

E.6 Les notifications push : `src/lib/push.ts`

```

Notifications.setNotificationHandler({
  // application ouverte : afficher la bannière, sans son
  handleNotification: async () => ({
    shouldPlaySound: false,
    shouldSetBadge: true,
    shouldShowBanner: true,
    shouldShowList: true,
  }),
});

export async function inscrirePush(): Promise<void> {
  try {
    if (!Device.isDevice || Constants.executionEnvironment === 'storeClient')
      return; // ni simulateur ni Expo Go
    if (Platform.OS === 'android')
      await Notifications.setNotificationChannelAsync('default', {
        name: 'Messages de l'école',
        importance: Notifications.AndroidImportance.HIGH,
      });
    const permission = await Notifications.getPermissionsAsync();
    const accordee =
      permission.granted ||
      (await Notifications.requestPermissionsAsync()).granted;
    if (!accordee) return;
    const { data } = await Notifications.getDevicePushTokenAsync(); // jeton FCM (Android) / APNs
  }
}

```

```
(iPhone)
  await envoyer('/notifications/jetons-push', 'POST', {
    jeton: String(data),
    plateforme: Platform.OS === 'ios' ? 'IOS' : 'ANDROID',
  });
} catch {
  /* le parent reçoit toujours SMS et email : le push est un complément */
}
}
```

(Point connu : l'iPhone natif envoie un jeton APNs alors que l'API parle à FCM ; les parents sur iPhone utilisent pour l'instant les notifications du site installé.) `push.web.ts` : versions vides pour le navigateur.

E.7 Un écran : l'accueil du parent (`(parent)/(onglets)/index.tsx`)

```
const RUBRIQUES = [ { href: '/resultats', libelle: 'Résultats', icone: '📊' }, ... ] as const;

export default function Accueil() {
  const styles = useStyles(creerStyles);
  const utilisateur = useUtilisateur();
  const enfants = useEnfants(); // liste + enfant sélectionné
  const messages = useMesNotifications(); // messages (avec cache hors ligne)
  return (
    <Ecran enChargement={messages.chargement}
      surRafrachir={() => { void messages.recharger(); void enfants.recharger(); }}> /* tirer
vers le bas */
      <Titre>Bonjour {utilisateur.prenoms}</Titre>
      <Etat chargement={...} erreur={...} vide={... && "Aucun enfant n'est rattaché à votre numéro..."}
surReessayer={enfants.recharger} />
      <ChoixEnfant /> /* boutons pour changer d'enfant */
      <View style={styles.grille}>
        {RUBRIQUES.map((r) => (
          <Pressable key={r.href} onPress={() => router.push(r.href)} accessibilityRole="button"
            style={({ pressed }) => [styles.tuile, pressed && styles.presse]}> /* style
différent pendant l'appui */
            <ContourDegrade rayon={16} style={{ flex: 1 }}>
              <View style={styles.tuileContenu}><Text style={styles.tuileIcône}>{r.icône}</Text>
              <Text style={styles.tuileTexte}>{r.libelle}</Text></View>
            </ContourDegrade>
          </Pressable>)))
        </View>
      {messages.liste.slice(0, 3).map((n) => <LigneNotification key={n.id} n={n} />)} /* 3 derniers
messages */
      ...
    </Ecran>
  );
}

const creerStyles = (couleurs: Couleurs) => StyleSheet.create({
  grille: { flexDirection: 'row', flexWrap: 'wrap', gap: 10 }, // en ligne, retour à la ligne,
  espacement
  tuile: { width: '31%', flexGrow: 1, minHeight: 100 }, // 3 tuiles par ligne
  tuileTexte: { fontSize: 14, fontWeight: '700', color: couleurs.titre, textAlign: 'center' },
  ...
});
```

`router.push('/resultats')` (Expo Router) ouvre un écran par-dessus ; `router.navigate` y va sans empiler.

E.8 Le scanner (`(personnel)/scanner.tsx`)

```
const [permission, demanderPermission] = useCameraPermissions(); // autorisation caméra
const dejaLu = useRef(false); // évite d'ouvrir 10 fois le même code
useFocusEffect(
  useCallback(() => {
    dejaLu.current = false;
    setActif(true);
    return () => setActif(false);
  }, []),
); // fonction de nettoyage : éteint la caméra en quittant
<CameraView
  style={StyleSheet.absoluteFill}
  facing="back"
  barcodeScannerSettings={{ barcodeTypes: ['qr'] }}
  onBarcodeScanned={({ data }) => {
    if (!dejaLu.current) ouvrir(data);
  }}
/>;
```

`ouvrir` extrait le code de l'étiquette (`codeEtiquette`) et ouvre `/appareil/[code]` ; on peut aussi taper le code court de 8 caractères.

E.9 Tous les fichiers du mobile

Fichier	Rôle
<code>app/connexion.tsx</code>	Parent : choix du pays, numéro, puis code SMS
<code>app/connexion-personnel.tsx</code>	Personnel : email et mot de passe
<code>app/(parent)/(onglets)/index.tsx</code>	Accueil : enfant, rubriques, 3 derniers messages
<code>app/(parent)/(onglets)/notifications.tsx</code>	Tous les messages
<code>app/(parent)/(onglets)/evenements.tsx</code>	Événements à venir
<code>app/(parent)/(onglets)/reglages.tsx</code>	Accès aux préférences des messages, déconnexion
<code>app/(parent)/notification/[id].tsx</code>	Un message (marqué lu à l'ouverture)
<code>app/(parent)/evenement/[id].tsx</code>	Un événement, réponse oui/non, pièce jointe
<code>app/(parent)/resultats.tsx</code>	Résultats publiés, bulletin PDF (téléchargé et partagé)
<code>app/(parent)/absences.tsx</code>	Absences, envoi d'un motif
<code>app/(parent)/comportement.tsx</code>	Comportements validés
<code>app/(parent)/paiements.tsx</code>	Paiements en attente et retard
<code>app/(parent)/appareils.tsx</code>	Appareils de l'enfant, déclaration de perte
<code>app/(parent)/preferences.tsx</code>	Canaux par type de message (interrupteurs)
<code>app/(personnel)/_layout.tsx</code> , <code>personnel.tsx</code>	Accueil du personnel
<code>app/(personnel)/scanner.tsx</code>	Lecture du QR code
<code>app/(personnel)/appareil/[code].tsx</code>	Appareil scanné : propriétaire, signalement (trouvé, confisqué...)

Fichier	Rôle
app/(personnel)/signaler-comportement.tsx	Signaler un comportement
components/ui.tsx	Briques : Ecran , Titre , Texte , Carte , Bouton , Champ , Message , Etat , ContourDegrade , ChoixTheme (bouton 🌞 / 🌙)...
components/entete.tsx	Entête en dégradé des écrans, avec le bouton 🌞 / 🌙
components/choix-enfant.tsx	Sélecteur d'enfant
components/consentement.tsx	Texte d'information à accepter
components/ligne-notification.tsx	Une ligne de message
lib/enfants.tsx	Contexte des enfants du parent et de l'enfant actif
lib/notifications.ts	Hook des messages avec cache hors ligne
lib/fichiers.ts / .web.ts	Télécharger puis partager un fichier (bulletin)
lib/format.ts (+ format.test.ts)	Formats français (dates, montants, monnaie), pays, code d'étiquette, fusion des messages
lib/types.ts	Types des réponses de l'API

Partie F — Le déploiement et la CI

F.1 docker-compose.prod.yml

```

name: suivi_eleve_prod
x-journaux:
  &journaux # « ancre » YAML : un bloc réutilisable (x- = extension ignorée par Docker)
  logging: { driver: json-file, options: { max-size: '10m', max-file: '5' } } # journaux limités à 5 x 10
  Mo

services:
  postgres: { image: postgres:17-alpine, ..., <<: *journaux } # <<: *journaux = insérer le bloc ancré
  redis:
    command: ['redis-server', '--appendonly', 'yes'] # Redis écrit aussi sur disque
  api:
    build: { context: ., dockerfile: api/Dockerfile } # image construite depuis le dépôt
    env_file: .env.production # secrets (jamais dans Git)
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}?
schema=public
      # « postgres » = nom du service : Docker le traduit en adresse sur son réseau interne
      REDIS_URL: redis://redis:6379
      WEB_ORIGIN: https://${DOMAINE}
    volumes: ['stockage:/donnees/stockage'] # photos persistantes
    depends_on:
      postgres: { condition: service_healthy } # attendre que la base réponde
  web:
    environment: { API_URL: http://api:3100/api, ... } # le site parle à l'API par le réseau interne
  caddy:
    ports: ['80:80', '443:443', '443:443/udp'] # seuls ports ouverts sur Internet (udp : HTTP/3)
    volumes: ['./deploy/Caddyfile:/etc/caddy/Caddyfile:ro', ...] # :ro = lecture seule

```

```

sauvegarde:
  build: { context: deploy/sauvegarde }
  volumes: [ './sauvegardes:/sauvegardes', 'stockage:/donnees/stockage:ro' ]
volumes: { postgres_data, redis_data, stockage, caddy_data, caddy_config }

```

Seul Caddy est exposé ; la base, Redis, l'API et le site ne sont joignables que **depuis l'intérieur** du réseau Docker.

F.2 web/Dockerfile

Même principe que C.10, avec une dernière étape qui ne copie que le **serveur autonome** :

```

COPY --from=construction --chown=node:node /app/web/.next/standalone ./
COPY --from=construction --chown=node:node /app/web/.next/static ./web/.next/static # fichiers
statiques
COPY --from=construction --chown=node:node /app/web/public ./web/public
CMD ["node", "web/server.js"]

```

F.3 deploy/Caddyfile

```

{$DOMAINE} {
    encode zstd gzip                # variable d'environnement : le nom du site
    handle /api/* {                 # compression des réponses
        reverse_proxy api:3100     # tout ce qui commence par /api...
    }                               # ... va à l'API
    handle {                        # le reste...
        reverse_proxy web:3001     # ... va au site
    }
    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains" # toujours HTTPS
        -Server                    # supprime l'en-tête qui donne le nom du serveur
    }
}

```

F.4 deploy/mettre-a-jour.sh

```

#!/bin/sh                                # « shebang » : programme qui exécute ce script
set -eu                                  # -e : arrêt à la première erreur ; -u : erreur si variable
inconnue
cd "$(dirname "$0")/.."                  # se placer à la racine du projet ($0 = chemin du script)
C="docker compose -f docker-compose.prod.yml --env-file .env.production"

echo "1/4 Sauvegarde de sécurité..."
$C exec -T sauvegarde sauvegarde.sh maintenant # exec : lance une commande dans le conteneur
echo "2/4 Récupération de la dernière version..."
git pull --ff-only                        # seulement si l'historique avance simplement
echo "3/4 Reconstruction et redémarrage (quelques minutes)..."
$C up -d --build                          # reconstruit les images modifiées et redémarre
docker image prune -f > /dev/null          # supprime les anciennes images (> /dev/null : sans
affichage)
echo "4/4 Vérification..."
DOMAINE=$(grep '^DOMAINE=' .env.production | cut -d= -f2) # $( ) : résultat d'une commande
essai=0
until curl -fsS "https://{$DOMAINE}/api/sante" > /dev/null 2>&1; do # jusqu'à ce que /api/sante réponde
   essai=$((essai + 1)) # $(( )) : calcul
    if [ "$essai" -ge 30 ]; then # -ge : supérieur ou égal (2 min 30)

```

```

    echo "L'API ne répond pas. Derniers messages :"; $C logs --tail 40 api; exit 1
  fi
  sleep 5
done
echo "Mise à jour terminée : $(git log -1 --format='%h %s')"
```

F.5 deploy/sauvegarde/sauvegarde.sh

- : "\${SAUVEGARDE_PHRASE:?...}" arrête le script si la phrase secrète manque.
- La phrase est écrite dans un fichier temporaire lisible par ce seul processus (`mktemp` , `chmod 600`), supprimé à la fin (`trap 'rm -f ...' EXIT`).
- `chiffrer()` : `gpg --symmetric --cipher-algo AES256` .
- `sauvegarder()` : `pg_dump -Fc | chiffrer ...` (le | envoie la sortie d'une commande à la suivante), écriture dans un fichier `.partiel` renommé à la fin (jamais de sauvegarde à moitié écrite), archive des photos, suppression des fichiers de plus de 14 jours (`find -mtime +14 -delete`).
- Sans argument : une boucle `while true` dort jusqu'à 2 h du matin puis sauvegarde. `restaurer.sh` fait l'inverse.

F.6 .github/workflows/ci.yml

```

name: CI
on:
  push: { branches: [main] }           # à chaque envoi sur main
  pull_request: { branches: [main] }
concurrency: { group: ci-${{ github.ref }}, cancel-in-progress: true } # annule l'exécution précédente
                                         devenue inutile
jobs:
  verifications:
    runs-on: ubuntu-latest              # machine Linux fournie par GitHub
    services:                           # conteneurs démarrés à côté
      postgres: { image: postgres:17-alpine, env: {...}, ports: ['5432:5432'], options: --health-cmd ... }
      redis: { image: redis:8-alpine, ... }
    env: { DATABASE_URL: ..., REDIS_URL: ..., JWT_SECRET: ... }
    steps:
      - uses: actions/checkout@v4        # récupère le code
      - uses: pnpm/action-setup@v4        # installe pnpm
      - uses: actions/setup-node@v4       # installe Node 22 (+ cache pnpm)
      - run: pnpm install --frozen-lockfile
      - run: prisma generate + prisma migrate deploy
      - run: pnpm format:check / pnpm lint / pnpm typecheck / pnpm test
      - run: pnpm --filter @suivi-eleve/api test:e2e
      - run: builds de l'API, du site, et « npx expo export --platform android »
  images:
    needs: verifications                # seulement si le premier travail a réussi
    steps: docker build de l'API, du site et de la sauvegarde
```

Partie G — Le voyage d'un message, de bout en bout

Pour tout relier, suivons une **libération anticipée** de la 6e A à 11 h 30.

1. **Navigateur** — La secrétaire ouvre `/annonces/nouvelle` . `proxy.ts` vérifie son cookie de session (D.2). La page serveur lit les classes par `lireApi('/classes')` (D.3) et affiche `ChampsAnnonce` (D.5).

2. **Formulaire** — Elle choisit « Libération anticipée », coche 6e A, saisit 11:30 et le motif, clique.
`FormulaireAction` demande confirmation puis appelle la Server Action `creerAnnonce` (D.5).
3. **Serveur du site** — `creerAnnonce` appelle `envoyerApi('/annonces', 'POST', {...})` : requête HTTP vers l'API avec le jeton et l'IP.
4. **API, entrée** — `JwtAuthGuard` vérifie le jeton, `RolesGuard` le rôle SECRETARIAT (C.4) ; le `ValidationPipe` contrôle le corps avec `CreerAnnonceDto` (C.5).
5. **API, logique** — `AnnoncesController.creer` → `AnnoncesService.creer` : vérifie la classe, calcule l'heure, crée l'annonce en base (Prisma → PostgreSQL), écrit le journal d'audit, puis `envoyer()` .
6. **Moteur** — `NotificationsService.notifier` (C.6) trouve les élèves de 6e A et leurs tuteurs, regroupe les frères et sœurs, rend les modèles (« les élèves de 6e A sont libérés à 11h30 (grève) »), crée pour chaque tuteur une ligne APPLICATION + SMS + EMAIL + PUSH, et met les envois en file BullMQ avec la priorité **URGENTE** (Redis).
7. **Réponse** — L'API répond tout de suite ; le site redirige vers `/annonces/[id]` , qui affiche le suivi et se rafraîchit.
8. **Travailleurs** — En arrière-plan, `EnvoiService.traiter` (C.6) envoie chaque SMS via Orange/Twilio, chaque email via Brevo, chaque push via FCM ou Web Push ; en cas d'échec : nouvel essai, puis bascule sur le second numéro.
9. **Parent** — Le téléphone sonne (SMS) ; la notification push s'affiche (`sw.js` ou `expo-notifications`). En l'ouvrant dans l'application, `marquerLue` passe la ligne APPLICATION à **LUE**.
10. **Retour** — Sur la page de suivi, la secrétaire voit « 18 familles, 12 ont lu » et la liste de celles qui n'ont pas encore lu.

Chaque étape correspond à un paragraphe de ce document : relisez-les dans cet ordre en ouvrant les fichiers, c'est la meilleure façon de comprendre comment l'application est conçue.